

《单片机原理及应用》PBL 项目合作学习作业

基于 C8051F310 单片机的科创集市项目设计

日期： 2026-06-19

专业班级： 光电信息科学与工程 202409

小组成员： 陈恪瑾（组长）、熊辰志、黄伟峰、安礼辰

小组队名： 闪电神经元队

作品名称： 基于 C8051F310 的反应力与节奏挑战训练仪

作品一句话描述：利用 C8051F310 开发板的数码管、矩阵键盘、LED 流水灯、蜂鸣器、外部触发键和 UART 串口，设计一台可现场互动的反应力与节奏挑战小游戏原型机，完成成绩显示、声光反馈、模式切换、键盘难度设置和串口状态输出。

一、思考题解答

1. C8051F310 单片机有哪些新特性？

C8051F310 是 Silicon Labs 推出的混合信号 SoC 型单片机，内部采用高速流水线结构的 CIP-51 内核，与 MCS-51 指令集完全兼容，但指令执行效率显著高于传统 8051。它集成了片内 Flash、片内 RAM、多个定时器、增强型 UART、SPI、SMBus/I2C、10 位 ADC、模拟比较器、片内温度传感器、片内高精度振荡器、上电复位、VDD 监视器、看门狗定时器和片内调试接口。与传统 8051 相比，它不仅保留了 8051 的软件生态和汇编开发方式，还把数据采集、控制、通信、调试和复位监控等功能集中在一颗芯片内，适合制作小型测控系统和教学综合实验项目。

从开发角度看，C8051F310 的重要新特性包括：第一，CIP-51 内核最高可达约 25MIPS，适合更高实时性任务；第二，端口 I/O 具有交叉开关配置机制，可把 UART、外部中断等数字外设灵活映射到引脚；第三，每个端口可配置为模拟输入、数字输入、开漏输出或推挽输出；第四，片上 ADC 可以直接采集模拟电压；第五，片内调试接口支持在线下载和调试，降低了实验开发难度。

2. C8051F310 的指令执行时间和传统 8051 单片机的区别是什么？

传统 8051 通常以机器周期为基本时序单位，一个机器周期一般等于 12 个振荡周期，多数指令需要 1 个或 2 个机器周期，即 12 个或 24 个时钟周期。C8051F310 的 CIP-51 内核采用流水线结构，指令时序直接以系统时钟周期计算。资料中指出，CIP-51

约 70% 的指令执行时间为 1 个或 2 个系统时钟周期，只有少数指令执行时间超过 4 个系统时钟周期。

因此，在相同或相近的系统时钟频率下，C8051F310 的指令执行速度明显高于传统 8051。对本项目而言，这种高速特性有利于实现 1ms 级定时、数码管动态刷新、键盘快速扫描、LED 节奏控制、软件 PWM 调光和串口发送等并行感较强的任务。

3. C8051F310 单片机定时器的时钟有哪些选择？

C8051F310 内部具有多个定时器/计数器。T0 和 T1 兼容传统 8051 定时器结构，可以工作在多种模式，并可选择系统时钟、由 CKCON 分频得到的时钟或外部计数输入。T2 和 T3 是增强型自动重装载定时器，可作为 16 位自动重装载定时器，也可拆分为两个 8 位自动重装载定时器。T2/T3 的时钟一般可以选择 SYSCLK、SYSCLK/12 或外部振荡源/8。

本项目使用 Timer0 作为周期性中断时间基准。开发板片内主频为 24.5MHz，程序将 Timer0 配置为 16 位定时器，并选择 SYSCLK/12 作为 Timer0 时钟。此时 Timer0 计数频率约为 2.041666MHz，1ms 约需要 2042 个计数，因此重装值设置为 0xF806，形成近似 1ms 的系统节拍。该节拍用于反应时间计数、键盘消抖、数码管刷新和游戏状态推进。

4. Configuration Wizard 提供图形化配置方法，请按照实验 3 需求配置 IO 口。

结合实验 3 键盘数码管需求和本项目功能，IO 口配置如下：

引脚/端口	配置方式	用途说明
P1.0–P1.7	推挽输出	数码管段选，输出 a、b、c、d、e、f、g、dp 段码
P0.6、P0.7	推挽输出	数码管位选 A、B，选择 4 位数码管中的当前一位
P2.0–P2.3	数字输出	4x4 矩阵键盘行扫描输出 KO.0–KO.3
P2.4–P2.7	数字输入	4x4 矩阵键盘列输入 KI.0–KI.3
P3.1	推挽输出	蜂鸣器控制，高电平有效
P3.2	普通数字输入/保留	本项目当前版本未使用，可作为后续模拟量扩展接口

P3.3	推挽输出	LED 流水灯 74HCT164 串行数据 DAT_IN
P3.4	推挽输出	LED 流水灯 74HCT164 移位时钟 CLK
P0.1	数字输入/外部中断	独立触发键 KINT，低电平有效，用于开始/下一轮
P0.4	UART0 TXD	串口发送开始、成绩和失败等状态信息到电脑
P0.5	UART0 RXD	UART 接收脚保留，当前程序未使用接收功能

Configuration Wizard 中需要注意：P1 设置为推挽输出以保证数码管段选驱动能力；P2 低四位作为矩阵键盘行扫描输出，高四位作为列输入并依靠弱上拉保持空闲高电平；P3.1、P3.3、P3.4 设置为推挽输出，用于蜂鸣器和 74HCT164 流水灯驱动。启用 UART0 后，P0.4 和 P0.5 作为 TXD/RXD 使用；启用交叉开关后，普通端口输出驱动才会正常工作。

5. C8051F310 单片机称为 SoC 的原因是什么？

SoC 即 System on Chip，含义是把一个完整系统所需的核心功能集成到同一芯片中。C8051F310 不仅有 CPU 内核，还集成了程序存储器 Flash、数据存储器 RAM、特殊功能寄存器、通用 I/O、定时器、UART/SPI/SMBus 通信接口、ADC、比较器、温度传感器、振荡器、复位电路、电源监测、看门狗和在线调试接口。它能够独立完成采集、运算、控制、显示、通信等任务。

本项目正体现了 SoC 的价值：反应力游戏需要定时、键盘输入、数码管显示、LED 驱动、蜂鸣器输出和 UART 通信，这些功能都可以围绕 C8051F310 一颗芯片完成，不需要再外接复杂的控制器。

6. 人工智能时代单片机可能的发展趋势。

人工智能时代的单片机将从传统的简单控制器，逐步发展为边缘智能节点。其趋势包括：第一，片上算力继续提升，部分 MCU 会集成轻量化 AI 加速单元，用于关键词识别、异常检测、姿态识别和传感数据分类；第二，模拟前端和传感接口更丰富，便于直接连接环境、人体和工业传感器；第三，低功耗能力更强，适合长时间运行的可穿戴设备、智能家居和物联网节点；第四，通信能力增强，更多 MCU 会集成蓝牙、Wi-Fi、LoRa 或更安全的有线通信；第五，开发工具更加智能化，AI 可以辅助生成配置代码、检查寄存器设置、分析波形和定位程序错误。

对于教学而言，未来单片机学习不仅要掌握寄存器、定时器和 I/O 控制，也要理解如何把单片机作为智能系统中的边缘执行端和数据采集端。本项目虽然不直接运行 AI 模型，但通过反应数据采集、键盘难度设置和串口状态输出，体现了边缘设备采集数据、做出反馈并与上位机协同的基本思路。

二、作品设计方案

1. 项目背景与意义

校园科创集市强调作品的可演示性、互动性和创造性。传统单片机实验通常围绕流水灯、数码管显示、键盘输入等单项功能展开，观众不容易直观看出系统设计的完整性。本项目把这些基础实验资源组合成一个可参与、可竞争、可展示成绩的互动挑战装置：反应力与节奏挑战训练仪。

在实际应用价值方面，反应力训练可用于课间娱乐、注意力训练和简单运动反应测试；节奏挑战可训练用户的节拍感和手眼协调能力。在教学意义方面，本作品覆盖了 C8051F310 开发板的主要数字 I/O 资源，能够综合考察小组成员对 I/O 配置、定时器中断、键盘扫描、动态显示、蜂鸣器控制、LED 串行驱动和 UART 通信的理解。相比单一实验，本项目更接近一个完整嵌入式原型机，适合用于科创集市现场动态演示。

2. 功能需求分析

核心功能如下：

1. 反应力测试：系统随机等待后显示目标键，玩家按下正确键后，数码管显示反应时间，单位为 ms。
2. 节奏挑战：LED 流水灯按节奏移动，玩家需要在随机指定的目标灯亮起时按 K0 得分，达到 20 分通关。
3. 模式切换：K15 在反应力测试和节奏挑战之间切换，KINT 用于开始游戏或进入下一轮。
4. 难度设置：在节奏模式下，K10、K11、K12 分别设置低、中、高三档难度，三档均要求在目标灯单点命中，区别为流水灯移动速度不同。
5. 数码管显示：反应模式显示目标数字和反应时间；节奏模式显示 LxSS，其中 x 为本局目标灯编号，SS 为当前分数。
6. 声光反馈：蜂鸣器提示开始、成功、失败和超时；LED 显示等待动画、节奏进度和成功动画。
7. UART 状态输出：通过 P0.4/P0.5 串口向电脑发送 START、SCORE 和 FAIL 等状态信息，便于调试和现场观察。

安全与交互约束如下：

1. 误按惩罚：目标出现前抢按或按错键时，本轮判定失败。
2. 按键消抖：矩阵键盘和 KINT 均进行软件消抖，避免一次机械按键被重复识别。
3. 结果保持：成功后保持反应时间或分数显示，失败后保持 FErr 提示，直到用户再次按 KINT 开始下一轮。
4. LED 调光：流水灯采用软件 PWM 降低亮度，避免现场演示时 LED 过亮影响观察。

3. 系统总体设计

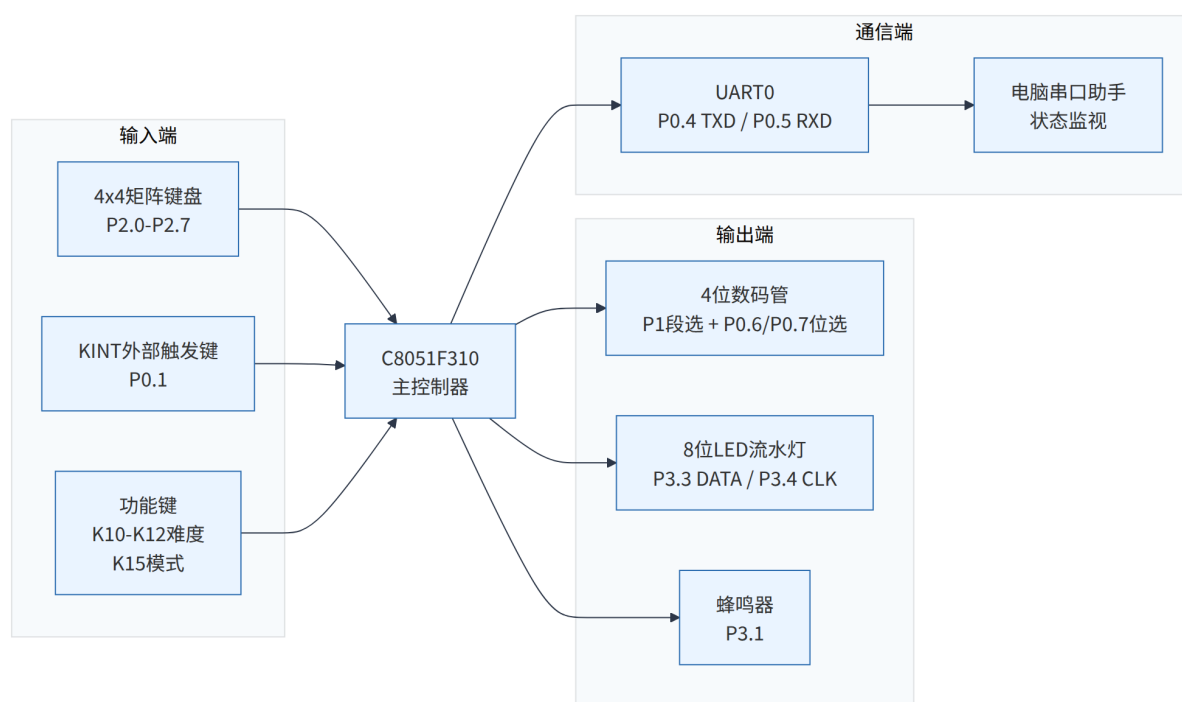


图 1: 系统总体硬件框图

系统采用“定时器中断 + 主循环状态机”的软件结构。Timer0 在 24.5MHz 片内时钟下通过 0xF806 重装值提供显示刷新、蜂鸣器和 LED 软件 PWM 服务；Timer2 作为 1ms 自动重载定时器，负责随机等待倒计时、反应时间计数和节奏模式的流水灯节拍。主循环根据状态变量执行待机、模式选择、难度设置、随机等待、显示目标、等待按键、成功反馈、失败反馈和串口状态输出等流程。

程序采用模块化工程结构。reaction_game.asm 作为顶层入口，统一包含公共定义和各个功能模块；reaction_game.inc 集中保存常量和内部 RAM 地址；其余模块分别承担向量表、初始化、主循环、游戏逻辑、中断、显示、键盘、随机数、LED、串口和查找表等职责。模块划分如表2所示。

表 2: 核心代码模块划分

文件	主要职责
Inc/reaction_game.inc	定义 Timer 重装值、状态常量、模式常量、按键常量、数码管索引和内部 RAM 固定地址。
Src/reaction_game.asm	顶层汇编入口，通过 <code>\$include</code> 组合公共定义和各个功能模块，并在文件末尾统一结束。
Src/vectors.asm	放置复位向量、INT0 向量、Timer0 向量和 Timer2 向量。
Src/init.asm	完成看门狗关闭、振荡器、端口、Timer0、Timer2、UART0 初始化，并进入主循环。
Src/main.asm	扫描 KINT 和矩阵键盘，根据当前状态分发到对应状态处理函数。
Src/game_logic.asm	实现反应模式、节奏模式、模式切换、难度设置、成功失败判定等核心状态机。
Src/isr.asm	Timer0 负责显示刷新、蜂鸣器和 LED PWM；Timer2 负责 1ms 计时、等待倒计时和节奏步进。
Src/display.asm	实现数码管动态扫描、段码查表、反应时间显示和节奏分数显示。
Src/keypad.asm	实现 4x4 矩阵键盘扫描、软件消抖、按键释放等待和 KINT 扫描。
Src/led_bar.asm	通过 P3.3/P3.4 驱动 74HCT164，完成 LED 流水灯输出和软件 PWM 调光。
Src/uart.asm	实现 UART 字符发送、字符串发送、成绩发送和失败状态发送。
Src/tables.asm	保存数码管段码表、节奏目标 LED 表和 UART 字符串。

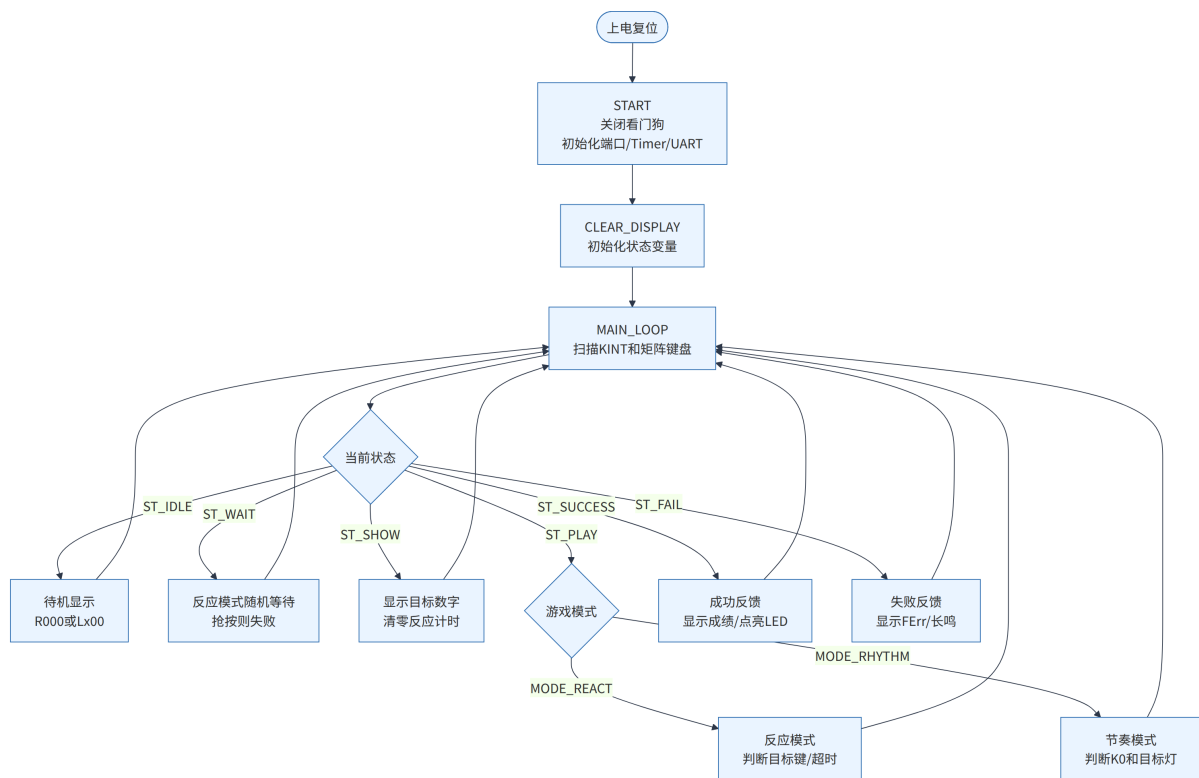


图 2: 系统软件主流程图

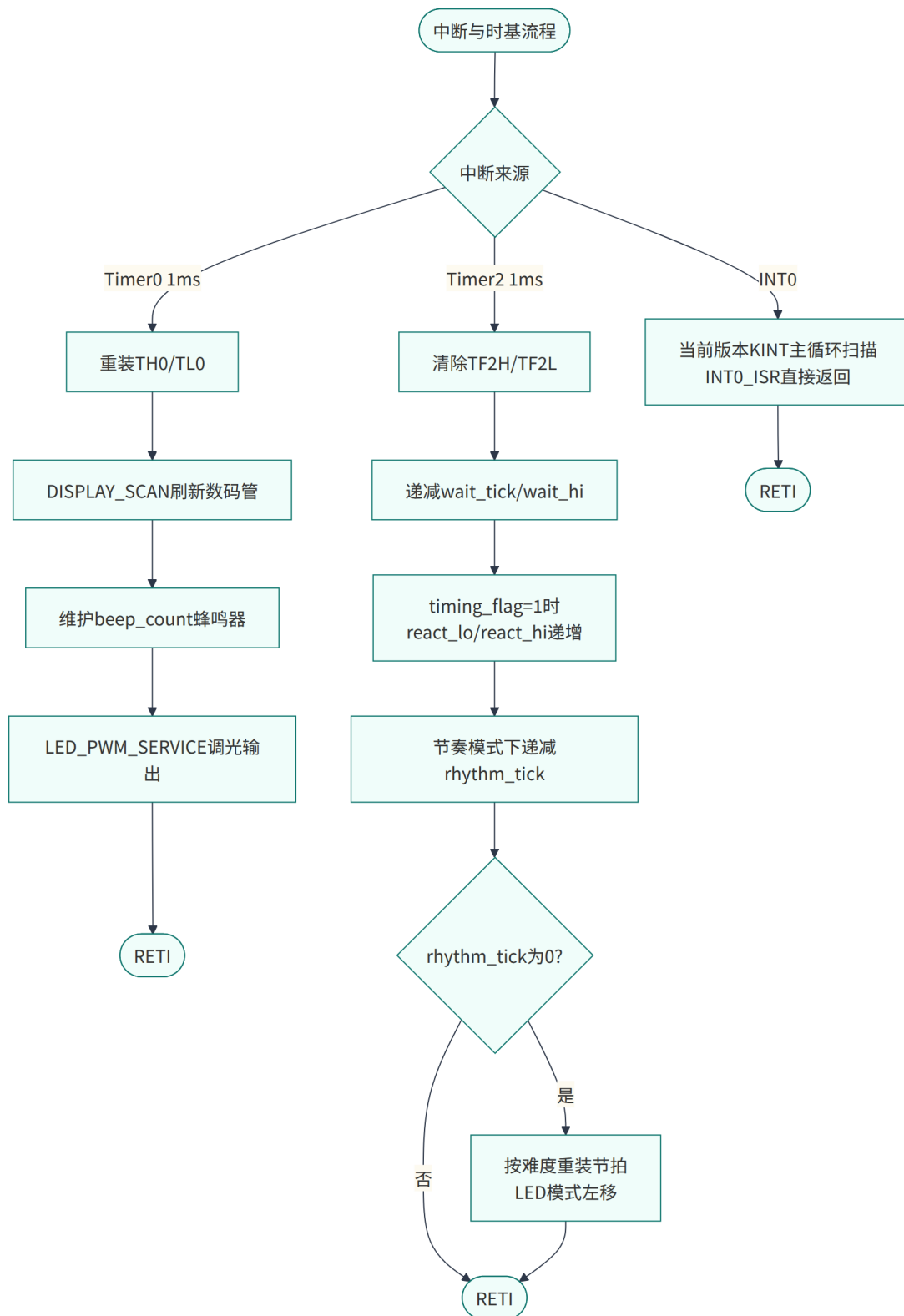


图 3: 中断与 1ms 时基流程图

4. 核心算法实现

算法一：随机目标生成与反应时间测量

本项目的随机数和反应计时均在汇编源代码中直接实现，完整代码见附录B。轻量级伪随机数由随机数与成绩辅助模块中的 MAKE_RANDOM 完成：程序读取历史随机种子，结合 Timer0 低字节进行异或、循环移位和常数扰动，得到新的 rand_seed。该随机数在游戏逻辑模块中被复用，一方面用于反应模式的随机等待时间，另一方面用于生成 0-9 目标数字；在节奏模式中，它还用于选择 D1-D8 目标灯。

反应时间测量的关键流程由游戏逻辑模块中的“显示目标”和“反应作答”两个状态完成。程序在目标数字送入数码管显示变量后，立即调用显示扫描，使目标可见边沿和计时开始尽量同步；随后清零 react_lo/react_hi 并启动 Timer2。Timer2 的 1ms 递增逻辑由中断模块维护，当 timing_flag 置位时持续累计 16 位反应时间。玩家按键后，主循环先锁存第一次扫描到的反应时间，再做消抖确认，因此消抖延时不会被计入最终成绩。反应时间转 4 位十进制显示由显示模块完成，相关代码均见附录B。

算法二：矩阵键盘扫描、消抖与节奏命中判定

矩阵键盘使用 P2 口。P2.0-P2.3 作为行扫描输出，对应 KO.0-KO.3；P2.4-P2.7 作为列输入，对应 KI.0-KI.3。程序每次把一行拉低，其余行置高，然后读取高四位列输入。如果某一列读到低电平，说明当前行列交点按键被按下。根据实验板按键示意图，内部键值采用“键值 = 列号 × 4 + 行号”的规则：第一列从上到下为 K0、K1、K2、K3，第二列为 K4-K7，第三列为 K8-K11，第四列为 K12-K15。

节奏挑战模式在开始时随机确定一个目标灯 D1-D8，数码管显示为 LxSS，其中 x 是目标灯编号，SS 是分数。玩家只有在目标灯亮起时按 K0 才得分；K10、K11、K12 用于低、中、高三档速度设置；K15 用于模式切换。三档难度均为单灯命中，区别是流水灯步进周期不同：低难度约 220ms/步，中难度约 160ms/步，高难度约 90ms/步。节奏模式达到 20 分判定成功。

矩阵键盘扫描、消抖和按键释放等待均由键盘模块实现，完整代码见附录B。其中 SCAN_KEY_RAW 负责逐行拉低 P2.0-P2.3 并读取 P2.4-P2.7 列输入，SCAN_KEY 在原始键值基础上加入软件消抖，WAIT_MATRIX_RELEASE 用于防止一次长按被重复计入。KINT 独立按键同样通过 SCAN_KINT 完成稳定按下和释放检测。

节奏命中判定由游戏逻辑模块中的 STATE_RHYTHM_PLAY 完成。程序要求玩家按下 K0，并且当前 led_pattern 必须等于目标灯模式 rhythm_pat，两者同时满足才增加 rhythm_score。LED 目标表由查找表模块提供；流水灯输出和软件 PWM 调光由 LED 模块完成；节奏速度控制由中断模块中的 Timer2 分支维护。低、中、高三档难度的步进周期分别约为 220ms、160ms 和 90ms，最终由 rhythm_tick 的重装值体现。上述实现对应的完整代码见附录B。

三、作品创新点和人机交互设计

1. 创新点阐述

创新点一：双模式挑战与键盘难度设置。普通反应力小游戏通常只有单一玩法，本项目在同一块 C8051F310 开发板上实现反应力测试和节奏挑战两种模式。K15 用于模式切换，K10-K12 用于节奏模式低、中、高三档速度设置。与固定速度的单一游戏相比，本作品既能测量反应时间，也能训练节奏感和手眼协调能力。该创新点使用状态机、矩阵键盘功能键分配、Timer2 节拍控制和数码管模式显示实现。

创新点二：多模态声光反馈。本项目不是单纯用数码管显示数字，而是把数码管、LED 流水灯和蜂鸣器组合为完整反馈系统。数码管负责显示分数、时间和目标键；LED 负责倒计时、节奏移动和成功动画；蜂鸣器负责开始、成功、失败和超时提示。与单一显示产品相比，用户无需阅读说明即可理解当前状态，现场演示效果更强。

创新点三：准确计时与柔和 LED 显示。反应模式不是简单在按键消抖后读取时间，而是在目标数字显示到数码管端口后立即启动 Timer2，并在第一次扫描到按键时锁存 16 位反应时间，误差控制在 10ms 以内。LED 流水灯通过 74HCT164 串行输出，同时加入软件 PWM 调光，使现场演示时灯光更柔和。该创新点使用 Timer2 自动重装载、16 位时间变量、二进制转 BCD 显示和 LED 占空比控制实现。

2. 人机交互设计

本作品实现三种主要交互方式。

第一种是矩阵键盘交互。用户可通过 4x4 键盘参与挑战，其中 K0-K9 用于反应模式数字目标输入，K0 也作为节奏模式命中键；K10、K11、K12 分别设置节奏模式低、中、高难度；K15 用于反应模式和节奏模式切换。系统检测到有效按键后，蜂鸣器短响，数码管更新显示，保证用户获得即时反馈。

第二种是外部触发键交互。P0.1 独立触发键 KINT 作为快速开始/下一轮按键，按下时为低电平。现场演示时，体验者只需按一下 KINT 即可开始当前模式；成功或失败后结果保持在数码管上，再次按 KINT 进入下一轮，提高了演示效率。

第三种是声光反馈交互。数码管显示当前模式、目标数字、目标灯编号、分数、反应时间或 FErr 错误提示；LED 流水灯显示等待动画和节奏进度；蜂鸣器用短响、长响区分开始、成功和失败。演示者可以先展示低难度，再切换高难度，让观众直接看到 LED 节奏加快。

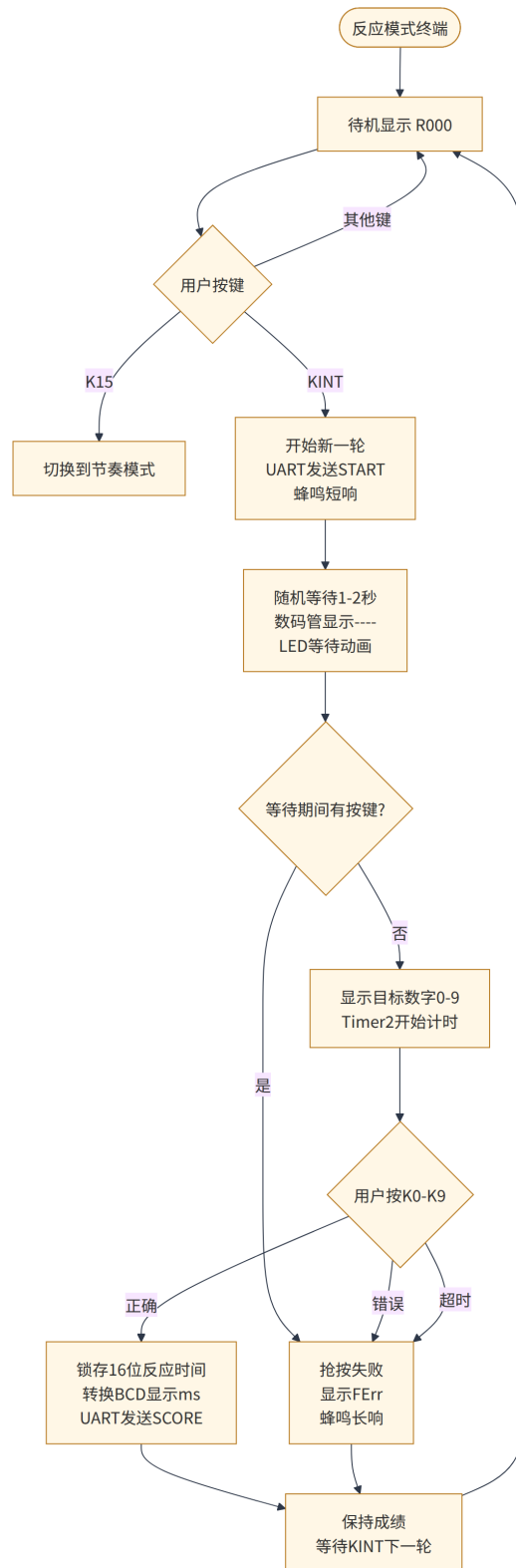


图 4: 反应模式终端流程图

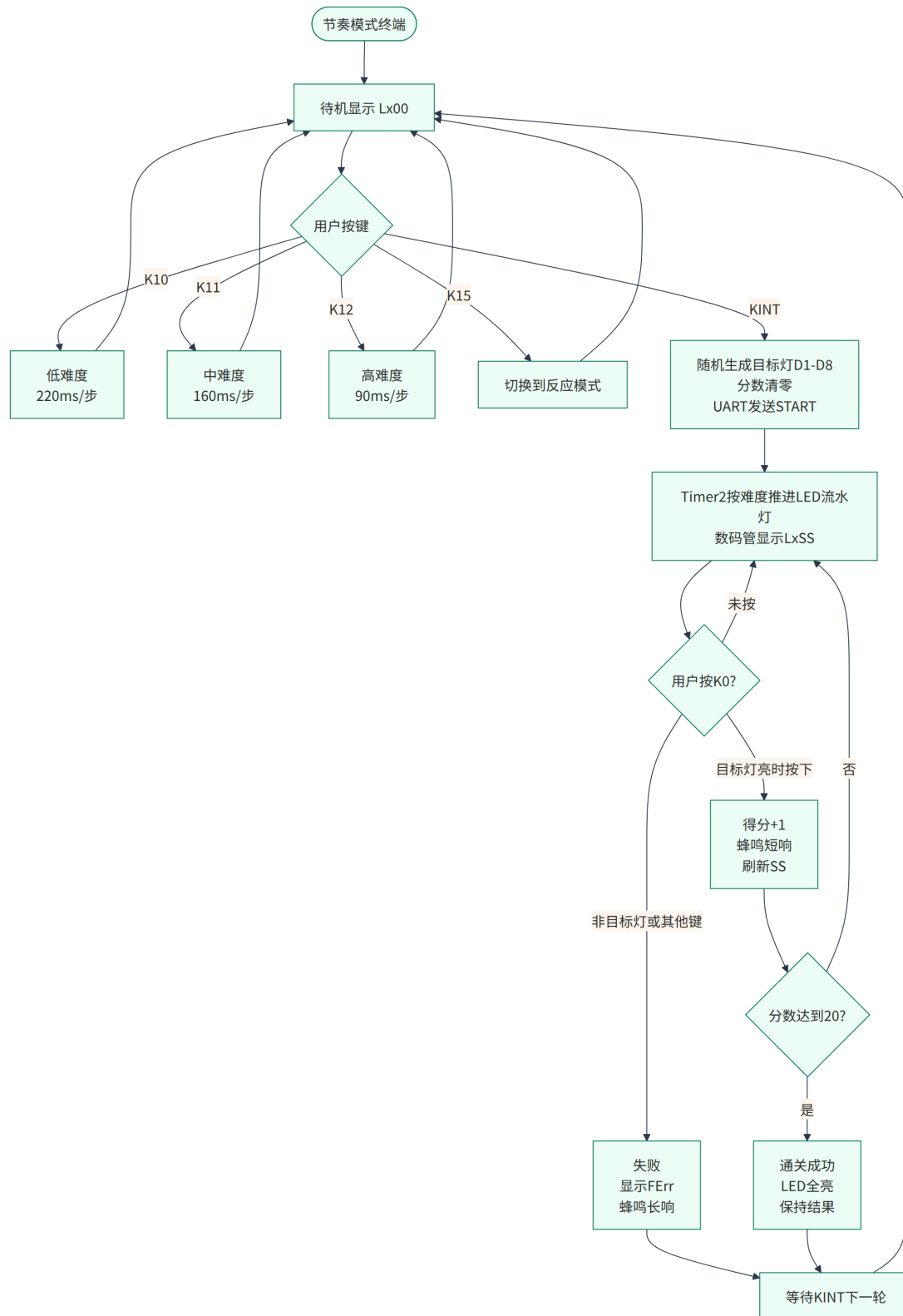


图 5: 节奏模式终端流程图

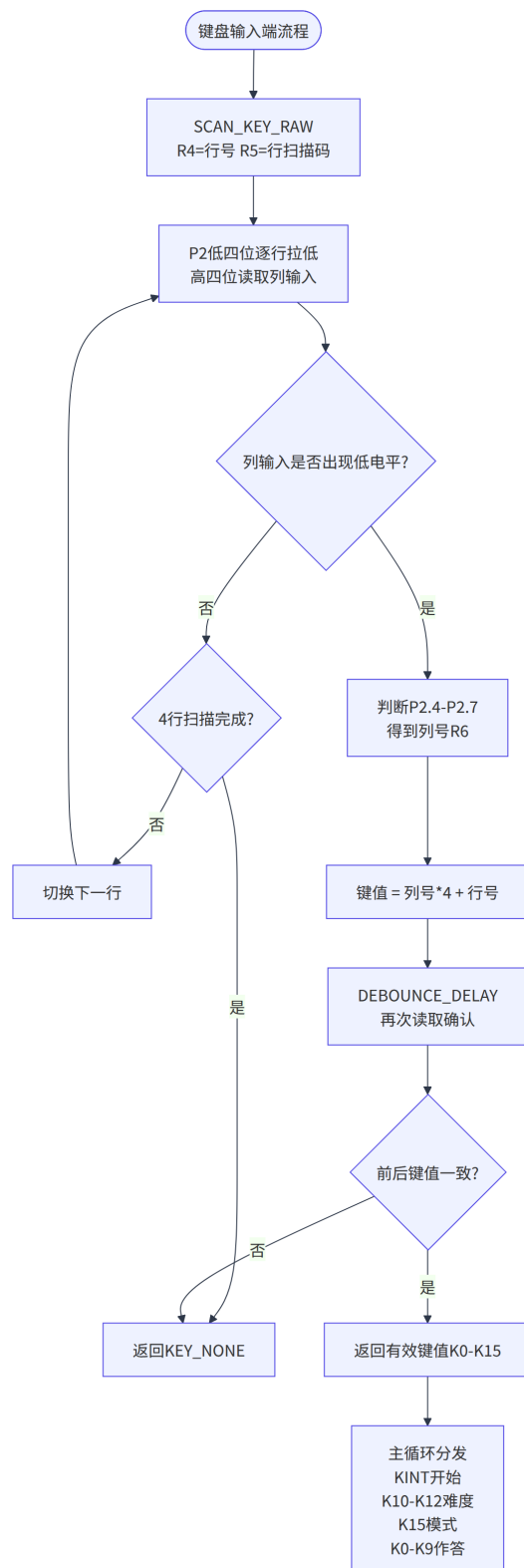


图 6: 键盘输入端流程图

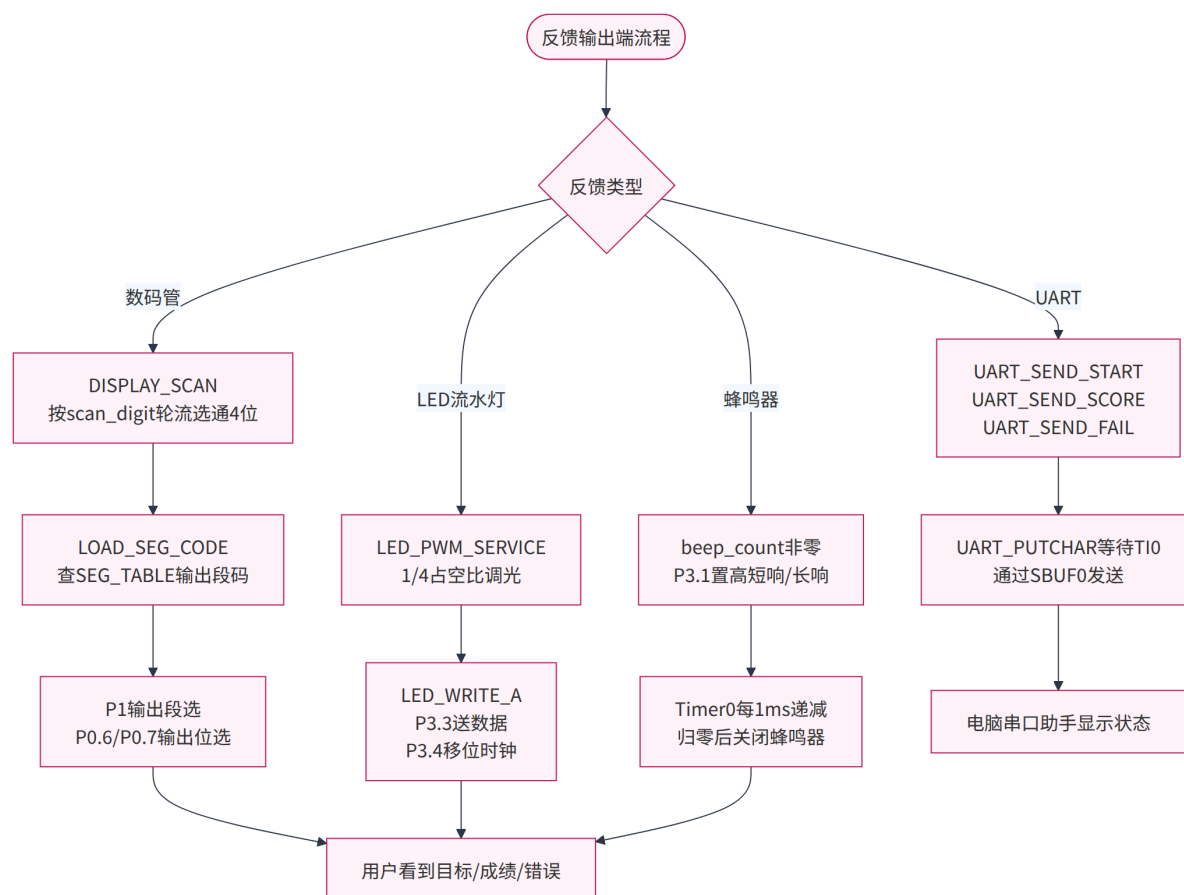


图 7: 反馈输出端流程图

现场演示中，演示者先说明规则，然后让观众按 K15 选择模式。反应模式下，观众按 KINT 开始，数码管随机显示 0-9 中的目标数字，观众按对应 K0-K9 数字键作答；成功时蜂鸣器短响并显示反应时间，失败时蜂鸣器长响并显示 FErr。节奏模式下，观众先用 K10-K12 选择难度，再按 KINT 开始；数码管显示 LxSS，其中 x 为本局随机目标灯编号，观众在对应 LED 亮起时按 K0 得分，达到 20 分通关。

四、小组讨论发言

按照模板要求，本节记录不少于 6 次小组讨论。由于聊天截图尺寸较大，采用分小节记录的方式，每次讨论单独说明主题、发言要点、形成结论，并插入对应截图。

1. 第一次讨论：项目选题比较

组长提出密码锁、抢答器、番茄钟、反应力游戏等候选方案。组员讨论后认为，科创集市作品需要具备现场互动性、可竞争性和直观反馈效果，因此最终确定“反应力与节奏挑战训练仪”作为项目主题。

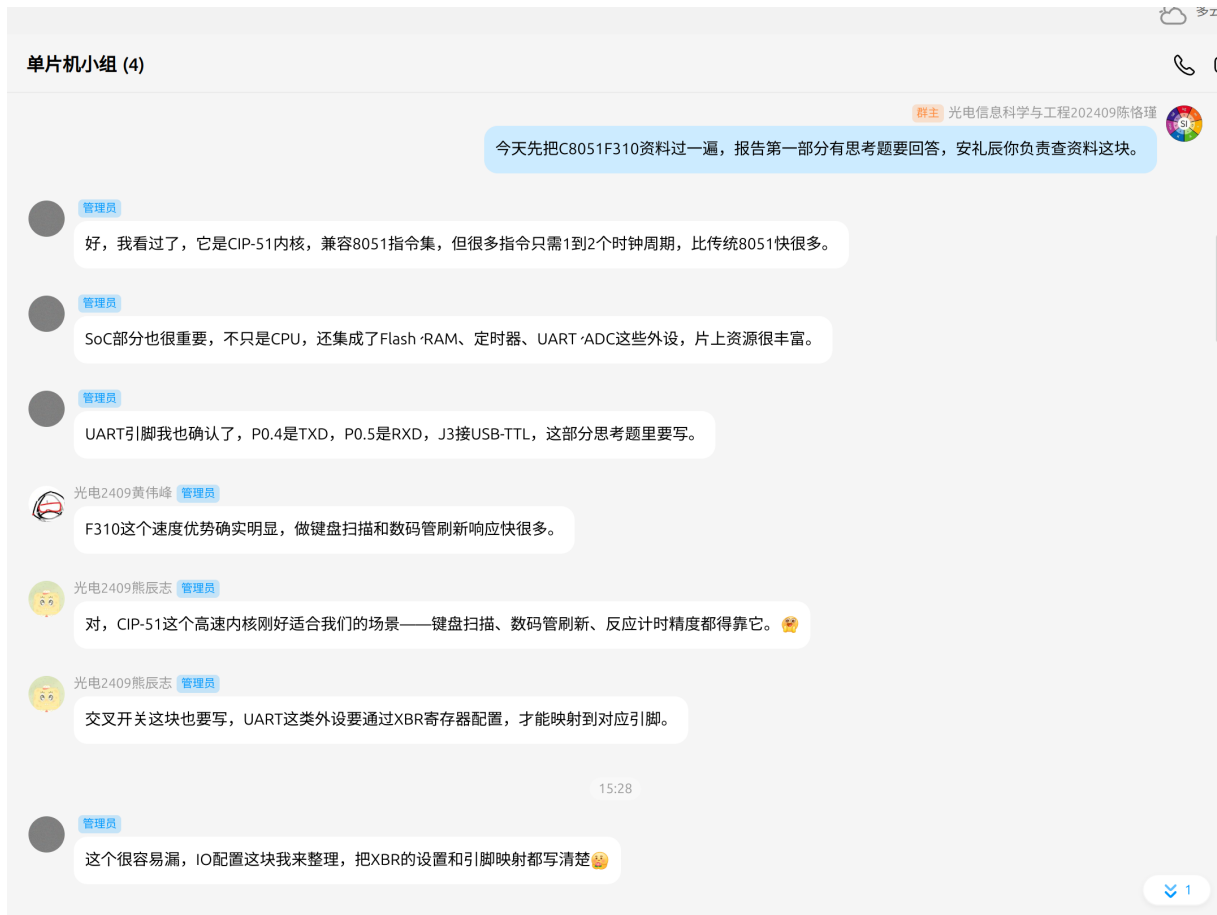


图 8: 第一次讨论截图 1



图 9: 第一次讨论截图 2

2. 第二次讨论：C8051F310 资料学习

本次讨论围绕 C8051F310 新特性、CIP-51 内核、SoC 集成资源、定时器时钟、交叉开关和 UART 接口展开。讨论结论是项目应尽量覆盖开发板上的数码管、矩阵键盘、LED 流水灯、蜂鸣器、KINT 按键和串口资源。

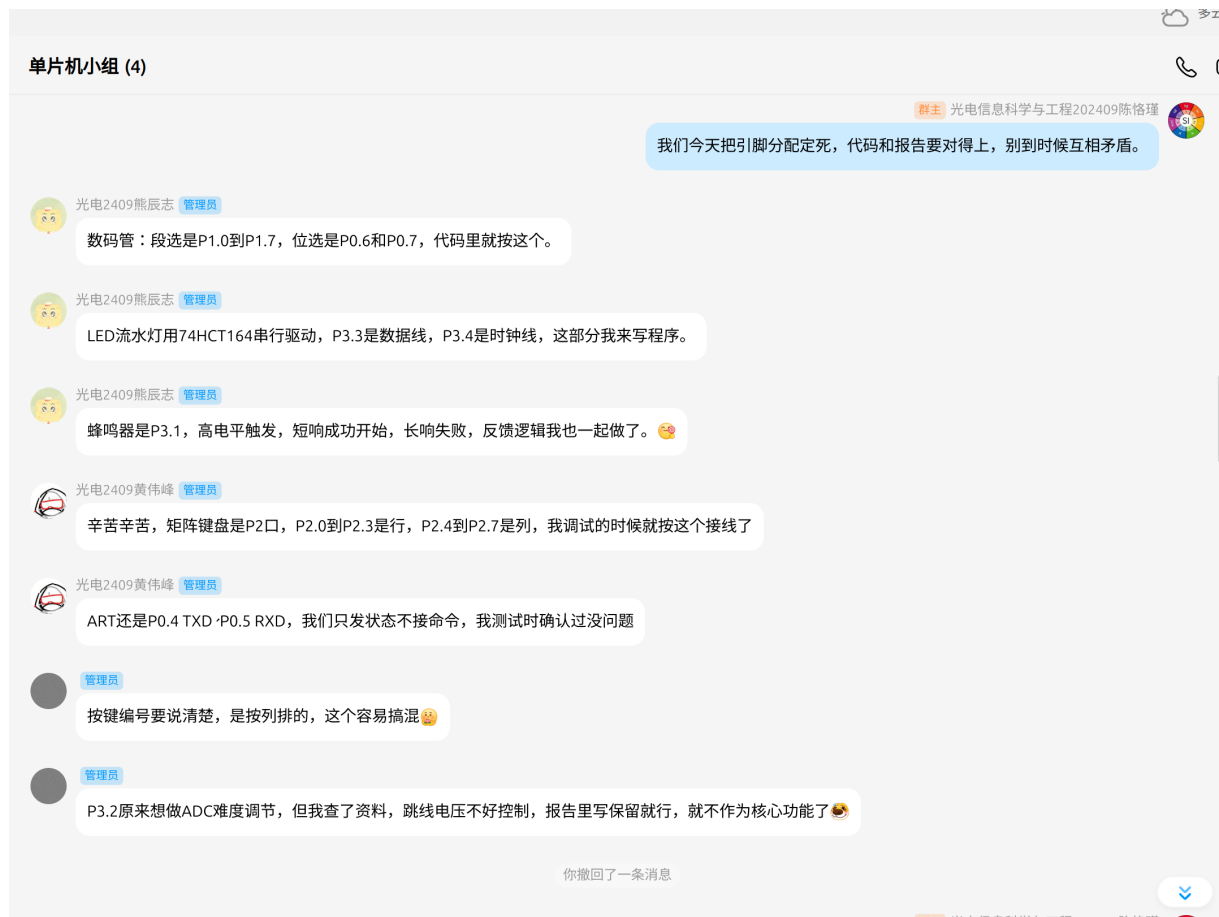


图 10: 第二次讨论截图 1



图 11: 第二次讨论截图 2

3. 第三次讨论：硬件资源分配

小组确认 P1 用于数码管段选，P0.6/P0.7 用于数码管位选，P2 连接 4x4 矩阵键盘，P3.1 驱动蜂鸣器，P3.3/P3.4 驱动 74HCT164 LED 流水灯，P0.4/P0.5 用于 UART 通信。该分配方案保证各硬件模块互不冲突，便于后续程序模块化实现。

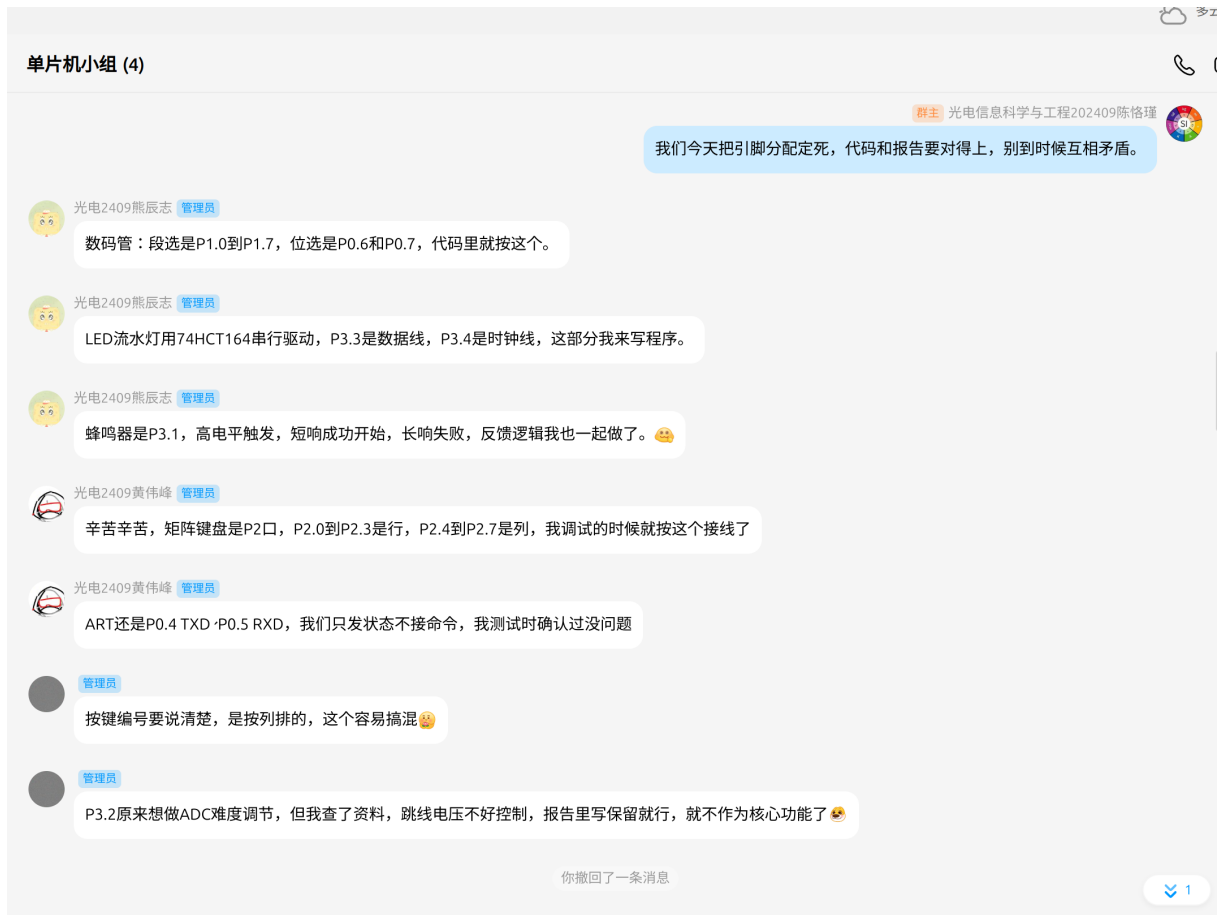


图 12: 第三次讨论截图 1



图 13: 第三次讨论截图 2

4. 第四次讨论：软件状态机设计

组长建议采用“待机—等待—显示目标—游戏—成功/失败”的状态机组织主流程。组员补充节奏模式需要目标灯、分数和速度档位，最终形成反应模式与节奏模式共用主循环、分别处理状态逻辑的双模式结构。



图 14: 第四次讨论截图

5. 第五次讨论：核心算法和交互优化

本次讨论重点是随机等待、反应时间锁存、矩阵键盘消抖和 LED 软件 PWM。小组决定在第一次扫描到按键时锁存反应时间，再进行消抖确认，减少因消抖延时导致的成绩误差；同时加入 K10-K12 三档节奏难度和 K15 模式切换。



图 15: 第五次讨论截图 1

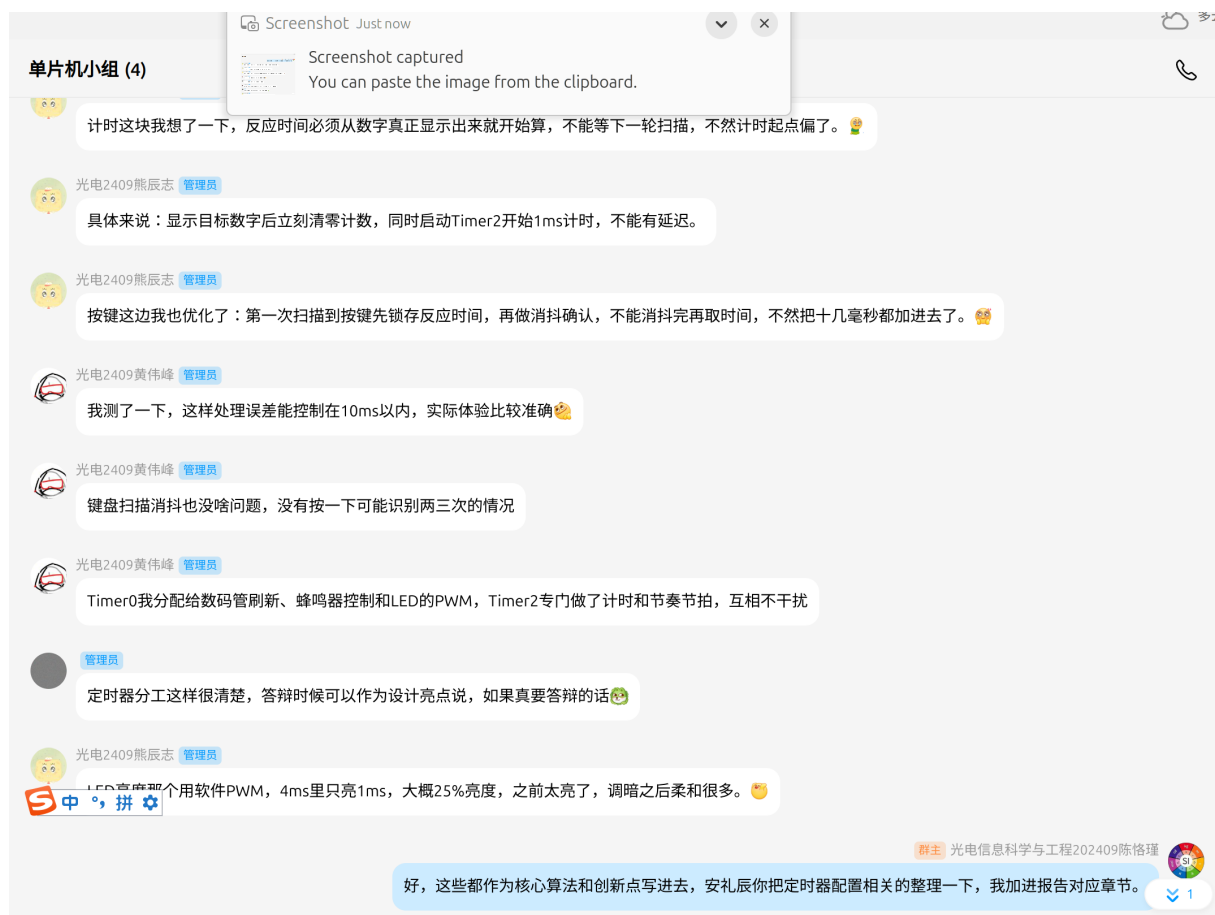


图 16: 第五次讨论截图 2

6. 第六次讨论：报告和代码整理

小组对照任务书评分点，补充三个创新点、两类以上人机交互方式、分工贡献说明和附录完整代码清单。代码采用多个 ASM 模块组织，报告中使用 Mermaid 流程图说明系统主流程、终端交互流程和中断时基流程。



图 17: 第六次讨论截图

五、项目总结和个人贡献表述

1. 项目预期成果和总结感悟

本项目预期完成一个基于 C8051F310 开发板的可运行原型机。上电后系统进入待机状态；用户可通过 K15 选择反应力测试或节奏挑战，通过 K10-K12 设置节奏难度；按下 KINT 后开始游戏；玩家根据数码管和 LED 提示按键；系统测量反应时间或统计节奏得分，并给出声光反馈；UART 向电脑输出 START、SCORE 和 FAIL 等状态信息。该作品把多个基础实验整合成一个完整互动系统，既有娱乐性，也能体现单片机综合设计能力。

通过本项目，小组成员能够进一步理解 C8051F310 作为 SoC 型单片机的特点，掌握定时器中断、矩阵键盘扫描、动态数码管显示、串口通信、LED 串行驱动和软件 PWM 调光的综合应用。项目也训练了团队分工、方案讨论、问题定位和现场演示表达能力。

2. 组长及组员分工和贡献描述

- 组长陈恪瑾：贡献比例 30%。负责确定项目主题，组织讨论，完成系统总体框图、软件流程、报告统稿和现场展示说明，并全程参与其他任务。
- 组员熊辰志：贡献比例 24%。主要负责设计键盘扫描、数码管显示、LED 流水灯、蜂鸣器提示和反应时间测量程序。
- 组员黄伟峰：贡献比例 23%。主要负责整理 ASM 代码并调试硬件设备，记录相关的测试数据并反馈，配合修改程序。
- 组员安礼辰：贡献比例 23%。主要负责查阅 C8051F310 资料，完成思考题解答，整理 IO 配置、UART 连接和定时器时钟相关内容。

一、UART 串口连接说明

开发板 UART 接口位于 J3，接口电平为 3.3V TTL。连接 USB-TTL 模块时，开发板 TXD 接 USB-TTL 的 RXD，开发板 RXD 接 USB-TTL 的 TXD，GND 必须共地。具体如下：

开发板信号	引脚/接口	USB-TTL 模块连接
TXD	P0.4 / J3-1	RXD
GND	J3-2	GND
RXD	P0.5 / J3-3	TXD

推荐串口参数为 9600 bps、8 位数据位、无校验、1 位停止位，即 9600, 8N1。程序使用 Timer1 方式 2 作为 UART0 波特率发生器，在 24.5MHz 系统时钟下令 Timer1 使用 SYSCLK 并设置 TH1=0xB0，可得到约 9570bps，接近 9600bps，实际串口助手按 9600bps 设置即可。

二、完整 ASM 代码清单

本附录收录本项目使用的完整汇编代码和头文件。

1. 芯片寄存器头文件

Listing 1: C8051F310.INC

```

1  ;-----
2  ;
3  ;
4  ;
5  ;
6  ; FILE NAME   : C8051F310.INC
7  ; TARGET MCUs : C8051F310, 'F311
8  ; DESCRIPTION : Register/bit definitions for the C8051F31x product family.
9  ;
10 ; REVISION 1.3
11 ;   -- added ESPIO and PSPIO
12 ; REVISION 1.2
13 ;   -- added VDMOCN (0xff)
14 ;
15 ; REVISION 1.1
16 ;   -- changed TARGET MCUs to 'F310, 'F311
17 ;   -- SPICFG --> SPIOCFG
18 ;   -- SPICKR --> SPIOCKR
19 ;   -- SPIDAT --> SPIODAT
20 ;   -- removed CLKMUL (0xb9)
21 ;   -- AMUXON --> AMXON
22 ;   -- AMUXOP --> AMXOP
23 ;
24 ;-----
25
26 ; BYTE Registers
27 PO      DATA 080H    ; PORT 0
28 SP      DATA 081H    ; STACK POINTER
29 DPL     DATA 082H    ; DATA POINTER - LOW BYTE
30 DPH     DATA 083H    ; DATA POINTER - HIGH BYTE
31 PCON    DATA 087H    ; POWER CONTROL
32 TCON    DATA 088H    ; TIMER CONTROL
33 TMOD    DATA 089H    ; TIMER MODE
34 TLO     DATA 08AH    ; TIMER 0 - LOW BYTE
35 TL1     DATA 08BH    ; TIMER 1 - LOW BYTE
36 TH0     DATA 08CH    ; TIMER 0 - HIGH BYTE
37 TH1     DATA 08DH    ; TIMER 1 - HIGH BYTE
38 CKCON   DATA 08EH    ; CLOCK CONTROL
39 PSCTL   DATA 08FH    ; PROGRAM STORE R/W CONTROL
40 P1      DATA 090H    ; PORT 1
41 TMR3CN   DATA 091H    ; TIMER 3 CONTROL
42 TMR3RL   DATA 092H    ; TIMER 3 RELOAD LOW
43 TMR3RLH  DATA 093H    ; TIMER 3 RELOAD HIGH
44 TMR3L    DATA 094H    ; TIMER 3 LOW BYTE
45 TMR3H    DATA 095H    ; TIMER 3 HIGH BYTE
46 SCON0    DATA 098H    ; SERIAL PORT 0 CONTROL

```

47	SBUF0	DATA	099H	; SERIAL PORT 0 BUFFER
48	CPT1CN	DATA	09AH	; COMPARATOR 1 CONTROL
49	CPT0CN	DATA	09BH	; COMPARATOR 0 CONTROL
50	CPT1MD	DATA	09CH	; COMPARATOR 1 MODE
51	CPT0MD	DATA	09DH	; COMPARATOR 0 MODE
52	CPT1MX	DATA	09EH	; COMPARATOR 1 MUX
53	CPT0MX	DATA	09FH	; COMPARATOR 0 MUX
54	P2	DATA	0A0H	; PORT 2
55	SPIOCFG	DATA	0A1H	; SPI0 CONFIGURATION
56	SPIOCKR	DATA	0A2H	; SPI0 CLOCK CONFIGURATION
57	SPIODAT	DATA	0A3H	; SPI0 DATA
58	POMDOUT	DATA	0A4H	; PORT 0 OUTPUT MODE
59	P1MDOUT	DATA	0A5H	; PORT 1 OUTPUT MODE
60	P2MDOUT	DATA	0A6H	; PORT 2 OUTPUT MODE
61	P3MDOUT	DATA	0A7H	; PORT 3 OUTPUT MODE
62	IE	DATA	0A8H	; INTERRUPT ENABLE
63	CLKSEL	DATA	0A9H	; CLOCK SOURCE SELECT
64	EMIOCN	DATA	0AAH	; EXTERNAL MEMORY INTERFACE CONTROL
65	P3	DATA	0B0H	; PORT 3
66	OSCXCN	DATA	0B1H	; EXTERNAL OSCILLATOR CONTROL
67	OSCICN	DATA	0B2H	; INTERNAL OSCILLATOR CONTROL
68	OSCICL	DATA	0B3H	; INTERNAL OSCILLATOR CALIBRATION
69	FLACL	DATA	0B5H	; FLASH ACCESS LIMIT
70	FLSCL	DATA	0B6H	; FLASH SCALE
71	FLKEY	DATA	0B7H	; FLASH LOCK & KEY
72	IP	DATA	0B8H	; INTERRUPT PRIORITY
73	AMXON	DATA	0BAH	; ADC0 MUX NEGATIVE CHANNEL SELECTION
74	AMXOP	DATA	0BBH	; ADC0 MUX POSITIVE CHANNEL SELECTION
75	ADCOCF	DATA	0BCH	; ADC0 CONFIGURATION
76	ADCOL	DATA	0BDH	; ADC0 DATA LOW
77	ADCOH	DATA	0BEH	; ADC0 DATA HIGH
78	SMB0CN	DATA	0C0H	; SMBUS CONTROL
79	SMB0CF	DATA	0C1H	; SMBUS CONFIGURATION
80	SMB0DAT	DATA	0C2H	; SMBUS DATA
81	ADCOGTL	DATA	0C3H	; ADC0 GREATER-THAN LOW
82	ADCOGTH	DATA	0C4H	; ADC0 GREATER-THAN HIGH
83	ADCOLTL	DATA	0C5H	; ADC0 LESS-THAN LOW
84	ADCOLTH	DATA	0C6H	; ADC0 LESS-THAN HIGH
85	TMR2CN	DATA	0C8H	; TIMER 2 CONTROL
86	TMR2RL	DATA	0CAH	; TIMER 2 RELOAD LOW
87	TMR2RLH	DATA	0CBH	; TIMER 2 RELOAD HIGH
88	TMR2L	DATA	0CCH	; TIMER 2 LOW BYTE
89	TMR2H	DATA	0CDH	; TIMER 2 HIGH BYTE
90	PSW	DATA	0D0H	; PROGRAM STATUS WORD
91	REF0CN	DATA	0D1H	; VOLTAGE REFERENCE 0 CONTROL
92	POSKIP	DATA	0D4H	; PORT 0 CROSSBAR SKIP
93	P1SKIP	DATA	0D5H	; PORT 1 CROSSBAR SKIP
94	P2SKIP	DATA	0D6H	; PORT 2 CROSSBAR SKIP
95	PCA0CN	DATA	0D8H	; PCA0 CONTROL
96	PCA0MD	DATA	0D9H	; PCA0 MODE
97	PCA0CPM0	DATA	0DAH	; PCA0 MODULE 0 MODE
98	PCA0CPM1	DATA	0DBH	; PCA0 MODULE 1 MODE
99	PCA0CPM2	DATA	0DCH	; PCA0 MODULE 2 MODE
100	PCA0CPM3	DATA	0DDH	; PCA0 MODULE 3 MODE
101	PCA0CPM4	DATA	0DEH	; PCA0 MODULE 4 MODE
102	ACC	DATA	0E0H	; ACCUMULATOR

```

103 XBRO    DATA  0E1H    ; DIGITAL CROSSBAR CONFIGURATION REGISTER 0
104 XBR1    DATA  0E2H    ; DIGITAL CROSSBAR CONFIGURATION REGISTER 1
105 ITO1CF   DATA  0E4H    ; INTO/INT1 CONFIGURATION
106 EIE1     DATA  0E6H    ; EXTERNAL INTERRUPT ENABLE 1
107 ADCOCN   DATA  0E8H    ; ADC 0 CONTROL
108 PCAOCPL1 DATA  0E9H    ; PCA0 MODULE 1 CAPTURE/COMPARE REGISTER LOW BYTE
109 PCAOCPH1 DATA  0EAH    ; PCA0 MODULE 1 CAPTURE/COMPARE REGISTER HIGH BYTE
110 PCAOCPL2 DATA  0EBH    ; PCA0 MODULE 2 CAPTURE/COMPARE REGISTER LOW BYTE
111 PCAOCPH2 DATA  0ECH    ; PCA0 MODULE 2 CAPTURE/COMPARE REGISTER HIGH BYTE
112 PCAOCPL3 DATA  0EDH    ; PCA0 MODULE 3 CAPTURE/COMPARE REGISTER LOW BYTE
113 PCAOCPH3 DATA  0EEH    ; PCA0 MODULE 3 CAPTURE/COMPARE REGISTER HIGH BYTE
114 RSTSRC   DATA  0EFH    ; RESET SOURCE
115 B        DATA  0F0H    ; B REGISTER
116 POMDIN   DATA  0F1H    ; PORT 0 INPUT MODE REGISTER
117 P1MDIN   DATA  0F2H    ; PORT 1 INPUT MODE REGISTER
118 P2MDIN   DATA  0F3H    ; PORT 2 INPUT MODE REGISTER
119 P3MDIN   DATA  0F4H    ; PORT 3 INPUT MODE REGISTER
120 EIP1     DATA  0F6H    ; EXTERNAL INTERRUPT PRIORITY 1
121 SPIOCN   DATA  0F8H    ; SPIO CONTROL
122 PCAOL    DATA  0F9H    ; PCA0 COUNTER REGISTER LOW BYTE
123 PCAOH    DATA  0FAH    ; PCA0 COUNTER REGISTER HIGH BYTE
124 PCAOCPL0 DATA  0FBH    ; PCA MODULE 0 CAPTURE/COMPARE REGISTER LOW BYTE
125 PCAOCPH0 DATA  0FCH    ; PCA MODULE 0 CAPTURE/COMPARE REGISTER HIGH BYTE
126 PCAOCPL4 DATA  0FDH    ; PCA MODULE 4 CAPTURE/COMPARE REGISTER LOW BYTE
127 PCAOCPH4 DATA  0FEH    ; PCA MODULE 4 CAPTURE/COMPARE REGISTER HIGH BYTE
128 VDMOCN   DATA  0FFH    ; VDD MONITOR CONTROL
129
130 ;-----
131 ;BIT DEFINITIONS
132 ;
133 ; TCON 88H
134 ITO      BIT    TCON.0   ; EXT. INTERRUPT 0 TYPE
135 IE0      BIT    TCON.1   ; EXT. INTERRUPT 0 EDGE FLAG
136 IT1      BIT    TCON.2   ; EXT. INTERRUPT 1 TYPE
137 IE1      BIT    TCON.3   ; EXT. INTERRUPT 1 EDGE FLAG
138 TRO      BIT    TCON.4   ; TIMER 0 ON/OFF CONTROL
139 TFO      BIT    TCON.5   ; TIMER 0 OVERFLOW FLAG
140 TR1      BIT    TCON.6   ; TIMER 1 ON/OFF CONTROL
141 TF1      BIT    TCON.7   ; TIMER 1 OVERFLOW FLAG
142
143 ; SCON0 0x98
144 RIO      BIT    SCON0.0   ; RECEIVE INTERRUPT FLAG
145 TIO      BIT    SCON0.1   ; TRANSMIT INTERRUPT FLAG
146 RB80     BIT    SCON0.2   ; RECEIVE BIT 8
147 TB80     BIT    SCON0.3   ; TRANSMIT BIT 8
148 RENO     BIT    SCON0.4   ; RECEIVE ENABLE
149 MCEO     BIT    SCON0.5   ; MULTIPROCESSOR COMMUNICATION ENABLE
150 SOMODE    BIT    SCON0.7   ; SERIAL MODE CONTROL BIT 0
151
152 ; IE 0xA8
153 EX0      BIT    IE.0      ; EXTERNAL INTERRUPT 0 ENABLE
154 ET0      BIT    IE.1      ; TIMER 0 INTERRUPT ENABLE
155 EX1      BIT    IE.2      ; EXTERNAL INTERRUPT 1 ENABLE
156 ET1      BIT    IE.3      ; TIMER 1 INTERRUPT ENABLE
157 ES0      BIT    IE.4      ; UART0 INTERRUPT ENABLE
158 ET2      BIT    IE.5      ; TIMER 2 INTERRUPT ENABLE

```

```

159 ESPIO    BIT    IE.6    ; SPI0 INTERRUPT ENABLE
160 EA      BIT    IE.7    ; GLOBAL INTERRUPT ENABLE
161
162 ; IP 0xB8
163 PX0     BIT    IP.0     ; EXTERNAL INTERRUPT 0 PRIORITY
164 PTO     BIT    IP.1     ; TIMER 0 PRIORITY
165 PX1     BIT    IP.2     ; EXTERNAL INTERRUPT 1 PRIORITY
166 PT1     BIT    IP.3     ; TIMER 1 PRIORITY
167 PS0     BIT    IP.4     ; UART0 PRIORITY
168 PT2     BIT    IP.5     ; TIMER 2 PRIORITY
169 PSPIO    BIT    IP.6     ; SPI0 INTERRUPT PRIORITY
170
171 ; SMBOCN 0xC0
172 SI      BIT    SMBOCN.0 ; SMBUS0 INTERRUPT FLAG
173 ACK     BIT    SMBOCN.1 ; ACKNOWLEDGE FLAG
174 ARBLOST BIT    SMBOCN.2 ; ARBITRATION LOST INDICATOR
175 ACKRQ   BIT    SMBOCN.3 ; ACKNOWLEDGE REQUEST
176 STO     BIT    SMBOCN.4 ; STOP FLAG
177 STA     BIT    SMBOCN.5 ; START FLAG
178 TXMODE  BIT    SMBOCN.6 ; TRANSMIT MODE INDICATOR
179 MASTER  BIT    SMBOCN.7 ; MASTER/SLAVE INDICATOR
180
181 ; TMR2CN 0xC8
182 T2XCLK  BIT    TMR2CN.0 ; TIMER 2 EXTERNAL CLOCK SELECT
183 TR2     BIT    TMR2CN.2 ; TIMER 2 ON/OFF CONTROL
184 T2SPLIT BIT    TMR2CN.3 ; TIMER 2 SPLIT MODE ENABLE
185 TF2LEN  BIT    TMR2CN.5 ; TIMER 2 LOW BYTE INTERRUPT ENABLE
186 TF2L    BIT    TMR2CN.6 ; TIMER 2 LOW BYTE OVERFLOW FLAG
187 TF2H    BIT    TMR2CN.7 ; TIMER 2 HIGH BYTE OVERFLOW FLAG
188
189 ; PSW 0xD0
190 P       BIT    PSW.0    ; ACCUMULATOR PARITY FLAG
191 F1      BIT    PSW.1    ; USER FLAG 1
192 OV      BIT    PSW.2    ; OVERFLOW FLAG
193 RS0     BIT    PSW.3    ; REGISTER BANK SELECT 0
194 RS1     BIT    PSW.4    ; REGISTER BANK SELECT 1
195 FO      BIT    PSW.5    ; USER FLAG 0
196 AC      BIT    PSW.6    ; AUXILIARY CARRY FLAG
197 CY      BIT    PSW.7    ; CARRY FLAG
198
199 ; PCAOCN 0xD8H
200 CCF0    BIT    PCAOCN.0 ; PCA0 MODULE 0 CAPTURE/COMPARE FLAG
201 CCF1    BIT    PCAOCN.1 ; PCA0 MODULE 1 CAPTURE/COMPARE FLAG
202 CCF2    BIT    PCAOCN.2 ; PCA0 MODULE 2 CAPTURE/COMPARE FLAG
203 CCF3    BIT    PCAOCN.3 ; PCA0 MODULE 3 CAPTURE/COMPARE FLAG
204 CCF4    BIT    PCAOCN.4 ; PCA0 MODULE 4 CAPTURE/COMPARE FLAG
205 CR      BIT    PCAOCN.6 ; PCA0 COUNTER RUN CONTROL
206 CF      BIT    PCAOCN.7 ; PCA0 COUNTER OVERFLOW FLAG
207
208 ; ADCOCN 0xE8H
209 ADCCM0  BIT    ADCOCN.0 ; ADC0 CONVERSION MODE SELECT 0
210 ADCCM1  BIT    ADCOCN.1 ; ADC0 CONVERSION MODE SELECT 1
211 ADCCM2  BIT    ADCOCN.2 ; ADC0 CONVERSION MODE SELECT 2
212 ADOWINT BIT    ADCOCN.3 ; ADC0 WINDOW COMPARE INTERRUPT FLAG
213 ADOBUSY BIT    ADCOCN.4 ; ADC0 BUSY FLAG
214 AD0INT  BIT    ADCOCN.5 ; ADC0 CONVERISION COMPLETE INTERRUPT FLAG

```

```

215 ADOTM    BIT    ADCOCN.6 ; ADCO TRACK MODE
216 ADOEN    BIT    ADCOCN.7 ; ADCO ENABLE
217
218 ; SPIOCN 0xF8H
219 SPIF      BIT    SPIOCN.7 ; SPI 0 INTERRUPT FLAG
220 WCOL      BIT    SPIOCN.6 ; SPI 0 WRITE COLLISION FLAG
221 MODF      BIT    SPIOCN.5 ; SPI 0 MODE FAULT FLAG
222 RXOVRN    BIT    SPIOCN.4 ; SPI 0 RX OVERRUN FLAG
223 NSSMD1    BIT    SPIOCN.3 ; SPI 0 SLAVE SELECT MODE 1
224 NSSMD0    BIT    SPIOCN.2 ; SPI 0 SLAVE SELECT MODE 0
225 TXBMT     BIT    SPIOCN.1 ; SPI 0 TRANSMIT BUFFER EMPTY
226 SPIEN     BIT    SPIOCN.0 ; SPI 0 SPI ENABLE

```

2. 项目公共定义

Listing 2: reaction_game.inc

```

1 ;-----
2 ; reaction_game.inc - shared constants and internal RAM layout
3 ;-----
4 $NOMOD51
5 $include (C8051F310.INC)
6
7 ; Internal oscillator on the C8051F310 EVM is 24.5 MHz.
8 ; Timer0/Timer2 use SYSCLK/12: 24.5 MHz / 12 = 2.041666 MHz.
9 ; 1 ms needs about 2042 counts, reload = 65536 - 2042 = F806H.
10 TIMER0_RELOAD_H EQU 0F8H
11 TIMER0_RELOAD_L EQU 006H
12
13 ; UART0 uses Timer1 mode 2 with Timer1 clock = SYSCLK.
14 ; Baud ~= 24.5 MHz / (32 * (256 - BOH)) = 9570 bps, close to 9600 8N1.
15 UART1_RELOAD    EQU 0B0H
16
17 ST_IDLE          EQU 00H
18 ST_WAIT          EQU 01H
19 ST_SHOW          EQU 02H
20 ST_PLAY          EQU 03H
21 ST_SUCCESS       EQU 04H
22 ST_FAIL          EQU 05H
23
24 MODE_REACT       EQU 00H
25 MODE_RHYTHM      EQU 01H
26
27 DIFF_LOW         EQU 00H
28 DIFF_MID         EQU 01H
29 DIFF_HIGH        EQU 02H
30
31 KEY_NONE         EQU 0FFH
32
33 BEEP_SHORT        EQU 18
34 BEEP_LONG         EQU 80
35
36 ; 7-seg codes for common-cathode display on this EVM, high active.
37 ; Index: 0..9,A,b,C,d,E,F, blank, dash, r, L

```

```

38 SEG_BLANK_IDX EQU 10H
39 SEG_DASH_IDX EQU 11H
40 SEG_R_IDX EQU 12H
41 SEG_L_IDX EQU 13H
42
43 ; Internal RAM variables. These fixed addresses match the original DSEG AT 30H
44 ; allocation so splitting the source does not change runtime state layout.
45 ms_sub EQU 30H
46 scan_digit EQU 31H
47 disp0 EQU 32H
48 disp1 EQU 33H
49 disp2 EQU 34H
50 disp3 EQU 35H
51 key_value EQU 36H
52 last_key EQU 37H
53 target_key EQU 38H
54 rand_seed EQU 39H
55 wait_tick EQU 3AH
56 wait_hi EQU 3BH
57 react_lo EQU 3CH
58 react_hi EQU 3DH
59 best_lo EQU 3EH
60 best_hi EQU 3FH
61 score_lo EQU 40H
62 score_hi EQU 41H
63 game_state EQU 42H
64 timing_flag EQU 43H
65 beep_count EQU 44H
66 led_pattern EQU 45H
67 led_pwm EQU 46H
68 show_timer EQU 47H
69 uart_temp EQU 48H
70 bcd_hund EQU 49H
71 bcd_tens EQU 4AH
72 bcd_ones EQU 4BH
73 bcd_thou EQU 4CH
74 kint_value EQU 4DH
75 conv_lo EQU 4EH
76 conv_hi EQU 4FH
77 game_mode EQU 50H
78 rhythm_tick EQU 51H
79 rhythm_score EQU 52H
80 rhythm_diff EQU 53H
81 rhythm_target EQU 54H
82 rhythm_pat EQU 55H

```

3. 顶层入口与中断向量

Listing 3: reaction_game.asm

```

1 ;-----
2 ; File      : reaction_game.asm
3 ; Target    : C8051F310 EVM
4 ; Project   : Reaction and Rhythm Challenge Trainer

```

```

5 ; Author   : PBL Group Template
6 ; Date    : 2026-06-17
7 ;
8 ; This top-level file keeps the original single-file build entry while the
9 ; implementation is organized like codes/Core/template:
10 ;   ../Inc/reaction_game.inc - shared constants and internal RAM symbols
11 ;   vectors.asm             - reset and interrupt vectors
12 ;   init.asm                - MCU and peripheral initialization
13 ;   main.asm                - main loop dispatcher
14 ;   game_logic.asm          - game state machine and mode handlers
15 ;   isr.asm                 - Timer0/Timer2/INT0 interrupt services
16 ;   display.asm             - 4-digit seven-segment display routines
17 ;   keypad.asm              - matrix keypad and KINT scanning
18 ;   random_score.asm        - random seed and best-score helpers
19 ;   led_bar.asm             - 74HCT164 LED output and PWM service
20 ;   uart.asm                - UART output helpers
21 ;   tables.asm              - ROM lookup tables and strings
22 ;
23 ; Hardware resource mapping from C8051F310EVM guide:
24 ;   4-digit 7-seg segments : P1.7..P1.0 = a,b,c,d,e,f,g,dp, high active
25 ;   4-digit 7-seg select  : P0.7=B, P0.6=A
26 ;   4x4 keypad rows       : P2.0..P2.3 output scan lines, K0.0..K0.3
27 ;   4x4 keypad columns    : P2.4..P2.7 input columns, KI.0..KI.3
28 ;   Buzzer                 : P3.1, high active
29 ;   LED array via 74HCT164 : P3.3 = DAT_IN, P3.4 = CLK, LED is active-low
30 ;   External trigger key   : P0.1, low active
31 ;   UART0                  : P0.4 = TXD, P0.5 = RXD, J3 uses 3.3V TTL level
32 ;-----
33
34 $include (../Inc/reaction_game.inc)
35
36 $include (vectors.asm)
37 $include (init.asm)
38 $include (main.asm)
39 $include (game_logic.asm)
40 $include (isr.asm)
41 $include (display.asm)
42 $include (keypad.asm)
43 $include (random_score.asm)
44 $include (led_bar.asm)
45 $include (uart.asm)
46 $include (tables.asm)
47
48         END

```

Listing 4: vectors.asm

```

1 ;-----
2 ; Interrupt vectors
3 ;-----
4         CSEG
5         ORG 0000H
6         LJMP START
7
8         ORG 0003H           ; External interrupt 0, P0.1 on this EVM design
9         LJMP INTO_ISR

```



```

10
11         ORG 000BH           ; Timer0 interrupt
12         LJMP TIMER0_ISR
13
14         ORG 002BH           ; Timer2 interrupt, exact 1 ms game time base
15         LJMP TIMER2_ISR

```

4. 初始化与主循环

Listing 5: init.asm

```

1  ;-----
2  ; init.asm - reset entry and MCU peripheral initialization
3  ;-----
4          ORG 0100H
5  START:
6          MOV SP, #70H
7          LCALL INIT_DEVICE
8          LCALL CLEAR_DISPLAY
9          MOV game_state, #ST_IDLE
10         MOV rand_seed, #5AH
11         MOV best_lo, #OFFH
12         MOV best_hi, #OFFH
13         MOV led_pattern, #0FEH
14         MOV led_pwm, #00H
15         MOV timing_flag, #00H
16         MOV last_key, #KEY_NONE
17         MOV beep_count, #00H
18         MOV show_timer, #00H
19         MOV kint_value, #00H
20         MOV wait_tick, #00H
21         MOV wait_hi, #00H
22         MOV game_mode, #MODE_REACT
23         MOV rhythm_tick, #00H
24         MOV rhythm_score, #00H
25         MOV rhythm_diff, #DIFF_LOW
26         MOV rhythm_target, #00H
27         MOV rhythm_pat, #0FEH
28         LCALL UART_BANNER
29
30         LJMP MAIN_LOOP
31
32  INIT_DEVICE:
33         LCALL PCA_INIT
34         LCALL OSC_INIT
35         LCALL PORT_IO_INIT
36         LCALL TIMER0_INIT
37         LCALL TIMER2_INIT
38         LCALL UART0_INIT
39         SETB EA
40         RET
41
42  PCA_INIT:
43         ANL PCA0MD, #0BFH           ; disable watchdog

```

```

44      MOV PCA0MD, #000H
45      RET
46
47 OSC_INIT:
48      MOV OSCICN, #083H      ; internal oscillator enabled, near max frequency
49      RET
50
51 PORT_IO_INIT:
52      ; P0.4/P0.5 UART, P0.6/P0.7 digit select push-pull.
53      MOV POMDOUT, #0DOH      ; P0.4 TXD, P0.6, P0.7 push-pull
54      MOV P1MDOUT, #0FFH      ; 7-seg segment pins push-pull
55      MOV P2MDOUT, #00FH      ; keypad rows push-pull, columns input
56      MOV P3MDOUT, #01AH      ; P3.1 buzzer, P3.3 data, P3.4 clock push-pull
57      MOV P3, #00H
58      MOV P2, #0FFH
59      MOV P1, #00H
60      MOV POSKIP, #00H
61      MOV P1SKIP, #00H
62      MOV P2SKIP, #00H
63      MOV XBR0, #01H          ; enable UART0 on crossbar
64      MOV XBR1, #40H          ; enable crossbar and weak pullups
65      ; External INTO on P0.1, low active. IT01CF low nibble selects INTO pin.
66      MOV IT01CF, #01H        ; INOSL=1 -> P0.1, low active default
67      SETB ITO                 ; edge triggered
68      CLR EXO                  ; KINT is handled by polling with debounce
69      RET
70
71 TIMERO_INIT:
72      ANL TMOD, #0FOH
73      ORL TMOD, #001H          ; Timer0 mode 1, 16-bit
74      ANL CKCON, #0F8H        ; SCA1..0=00, TOM=0, Timer0 uses SYSCLK/12
75      MOV TH0, #TIMERO_RELOAD_H
76      MOV TLO, #TIMERO_RELOAD_L
77      SETB ET0
78      SETB TR0
79      RET
80
81 TIMER2_INIT:
82      CLR TR2
83      MOV TMR2CN, #00H         ; 16-bit auto-reload timer mode
84      ANL CKCON, #0CFH        ; Timer2 high/low bytes use SYSCLK/12
85      MOV TMR2RL, #TIMERO_RELOAD_L
86      MOV TMR2RH, #TIMERO_RELOAD_H
87      MOV TMR2L, #TIMERO_RELOAD_L
88      MOV TMR2H, #TIMERO_RELOAD_H
89      CLR TF2L
90      CLR TF2H
91      SETB ET2
92      SETB TR2
93      RET
94
95 UART0_INIT:
96      ; UART0 mode 1, variable baud. Timer1 mode 2 for 9600 baud approximate at 24.5MHz.
97      ANL TMOD, #00FH
98      ORL TMOD, #020H          ; Timer1 mode 2
99      ORL CKCON, #008H        ; T1M=1, Timer1 uses SYSCLK

```

```

100      MOV TH1, #UART1_RELOAD
101      MOV TL1, #UART1_RELOAD
102      MOV SCON0, #050H          ; 8-bit UART, REN enabled
103      SETB TR1
104      SETB TIO
105      RET

```

Listing 6: main.asm

```

1  ;-----
2  ; main.asm - main loop dispatcher
3  ;-----
4  MAIN_LOOP:
5      LCALL SCAN_KINT
6      MOV kint_value, A
7      JZ MAIN_NO_KINT
8      LCALL HANDLE_KINT
9
10 MAIN_NO_KINT:
11     MOV A, game_state
12     CJNE A, #ST_PLAY, MAIN_SCAN_DEBOUNCED
13     LCALL SCAN_KEY_RAW          ; in play state, capture the first key edge quickly
14     SJMP MAIN_KEY_SCANNED
15 MAIN_SCAN_DEBOUNCED:
16     LCALL SCAN_KEY
17 MAIN_KEY_SCANNED:
18     MOV key_value, A
19     MOV A, key_value
20     CJNE A, #KEY_NONE, MAIN_HAS_MATRIX_KEY
21     SJMP MAIN_NO_FUNC_KEY
22
23 MAIN_HAS_MATRIX_KEY:
24     CJNE A, #OFH, MAIN_CHECK_DIFF_KEY
25     LCALL HANDLE_MODE_KEY
26     SJMP MAIN_NO_FUNC_KEY
27
28 MAIN_CHECK_DIFF_KEY:
29     LCALL HANDLE_DIFFICULTY_KEY
30
31 MAIN_NO_FUNC_KEY:
32     MOV A, game_state
33     CJNE A, #ST_IDLE, MAIN_NOT_IDLE
34     LCALL STATE_IDLE
35     SJMP MAIN_LOOP
36
37 MAIN_NOT_IDLE:
38     CJNE A, #ST_WAIT, MAIN_NOT_WAIT
39     LCALL STATE_WAIT
40     SJMP MAIN_LOOP
41
42 MAIN_NOT_WAIT:
43     CJNE A, #ST_SHOW, MAIN_NOT_SHOW
44     LCALL STATE_SHOW
45     SJMP MAIN_LOOP
46
47 MAIN_NOT_SHOW:

```

```

48         CJNE A, #ST_PLAY, MAIN_NOT_PLAY
49         LCALL STATE_PLAY
50         SJMP MAIN_LOOP
51
52 MAIN_NOT_PLAY:
53         CJNE A, #ST_SUCCESS, MAIN_NOT_SUCCESS
54         LCALL STATE_SUCCESS
55         SJMP MAIN_LOOP
56
57 MAIN_NOT_SUCCESS:
58         CJNE A, #ST_FAIL, MAIN_LOOP
59         LCALL STATE_FAIL
60         SJMP MAIN_LOOP

```

5. 游戏状态机与中断服务

Listing 7: game_logic.asm

```

1  ;-----
2  ; game_logic.asm - game state machine and mode handlers
3  ;-----
4  STATE_IDLE:
5      MOV A, game_mode
6      CJNE A, #MODE_RHYTHM, IDLE_SHOW_REACT
7      MOV disp3, #SEG_L_IDX
8      MOV A, rhythm_diff
9      INC A
10     MOV disp2, A
11     MOV disp1, #00H
12     MOV disp0, #00H
13     RET
14 IDLE_SHOW_REACT:
15     MOV disp3, #SEG_R_IDX
16     MOV disp2, #SEG_BLANK_IDX
17     MOV disp1, #00H
18     MOV disp0, #00H
19     RET
20
21 HANDLE_KINT:
22     MOV A, game_state
23     CJNE A, #ST_IDLE, HK_NOT_IDLE
24     LCALL NEW_ROUND
25     RET
26 HK_NOT_IDLE:
27     CJNE A, #ST_SUCCESS, HK_NOT_SUCCESS
28     LCALL NEW_ROUND
29     RET
30 HK_NOT_SUCCESS:
31     CJNE A, #ST_FAIL, HK_DONE
32     LCALL NEW_ROUND
33 HK_DONE:
34     RET
35
36 HANDLE_MODE_KEY:

```

```

37         MOV A, game_state
38         CJNE A, #ST_IDLE, HMK_NOT_IDLE
39         SJMP HMK_TOGGLE
40 HMK_NOT_IDLE:
41         CJNE A, #ST_SUCCESS, HMK_NOT_SUCCESS
42         SJMP HMK_TOGGLE
43 HMK_NOT_SUCCESS:
44         CJNE A, #ST_FAIL, HMK_DONE
45 HMK_TOGGLE:
46         MOV A, game_mode
47         XRL A, #01H
48         MOV game_mode, A
49         MOV game_state, #ST_IDLE
50         MOV timing_flag, #00H
51         MOV wait_tick, #00H
52         MOV wait_hi, #00H
53         MOV led_pattern, #0FFH
54         LCALL LED_WRITE_A
55         MOV beep_count, #BEEP_SHORT
56         LCALL WAIT_MATRIX_RELEASE
57 HMK_DONE:
58         RET
59
60 HANDLE_DIFFICULTY_KEY:
61         MOV A, game_mode
62         CJNE A, #MODE_RHYTHM, HDK_DONE
63         MOV A, game_state
64         CJNE A, #ST_IDLE, HDK_NOT_IDLE
65         SJMP HDK_CHECK_KEY
66 HDK_NOT_IDLE:
67         CJNE A, #ST_SUCCESS, HDK_NOT_SUCCESS
68         SJMP HDK_CHECK_KEY
69 HDK_NOT_SUCCESS:
70         CJNE A, #ST_FAIL, HDK_DONE
71 HDK_CHECK_KEY:
72         MOV A, key_value
73         CJNE A, #OAH, HDK_NOT_LOW
74         MOV rhythm_diff, #DIFF_LOW
75         SJMP HDK_SET_DONE
76 HDK_NOT_LOW:
77         CJNE A, #OBH, HDK_NOT_MID
78         MOV rhythm_diff, #DIFF_MID
79         SJMP HDK_SET_DONE
80 HDK_NOT_MID:
81         CJNE A, #OCH, HDK_DONE
82         MOV rhythm_diff, #DIFF_HIGH
83 HDK_SET_DONE:
84         MOV game_state, #ST_IDLE
85         MOV rhythm_score, #00H
86         MOV led_pattern, #0FFH
87         LCALL LED_WRITE_A
88         MOV beep_count, #BEEP_SHORT
89         LCALL WAIT_MATRIX_RELEASE
90 HDK_DONE:
91         RET
92

```

```

93 NEW_ROUND:
94     MOV A, game_mode
95     CJNE A, #MODE_RHYTHM, NEW_REACTION_ROUND
96     LCALL NEW_RHYTHM_ROUND
97     RET
98
99 NEW_REACTION_ROUND:
100    LCALL MAKE_RANDOM
101    ; Random wait is about 1..2 seconds.
102    ; Base = 1000 ms = 03E8H. Add rand_seed four times:
103    ; 03E8H + 0..1020 ms = about 1.0..2.02 s.
104    MOV wait_hi, #03H
105    MOV wait_tick, #0E8H
106    MOV R7, #04H
107 NR_WAIT_ADD:
108    MOV A, wait_tick
109    ADD A, rand_seed
110    MOV wait_tick, A
111    JNC NR_WAIT_NO_CARRY
112    INC wait_hi
113 NR_WAIT_NO_CARRY:
114    DJNZ R7, NR_WAIT_ADD
115    MOV disp3, #SEG_DASH_IDX
116    MOV disp2, #SEG_DASH_IDX
117    MOV disp1, #SEG_DASH_IDX
118    MOV disp0, #SEG_DASH_IDX
119    MOV game_state, #ST_WAIT
120    MOV timing_flag, #00H
121    MOV beep_count, #BEEP_SHORT
122    LCALL UART_SEND_START
123    RET
124
125 NEW_RHYTHM_ROUND:
126    LCALL MAKE_RANDOM
127    MOV A, rand_seed
128    ANL A, #07H
129    MOV rhythm_target, A
130    LCALL LOAD_RHYTHM_PATTERNS
131    MOV rhythm_score, #00H
132    MOV A, rhythm_diff
133    CJNE A, #DIFF_LOW, NRR_NOT_LOW
134    MOV rhythm_tick, #220
135    SJMP NRR_SPEED_DONE
136 NRR_NOT_LOW:
137    CJNE A, #DIFF_HIGH, NRR_NORMAL_SPEED
138    MOV rhythm_tick, #90
139    SJMP NRR_SPEED_DONE
140 NRR_NORMAL_SPEED:
141    MOV rhythm_tick, #160
142 NRR_SPEED_DONE:
143    MOV led_pattern, #0FEH
144    MOV timing_flag, #00H
145    MOV wait_tick, #00H
146    MOV wait_hi, #00H
147    LCALL LED_WRITE_A
148    LCALL SHOW_RHYTHM_SCORE

```

```

149     MOV game_state, #ST_PLAY
150     MOV beep_count, #BEEP_SHORT
151     LCALL UART_SEND_START
152     RET
153
154 STATE_WAIT:
155     MOV disp3, #SEG_DASH_IDX
156     MOV disp2, #SEG_DASH_IDX
157     MOV disp1, #SEG_DASH_IDX
158     MOV disp0, #SEG_DASH_IDX
159     ; A key pressed before target appears is a false start.
160     MOV A, key_value
161     CJNE A, #KEY_NONE, WAIT_FALSE_START
162     MOV A, wait_tick
163     ORL A, wait_hi
164     JZ WAIT_DONE
165     RET
166 WAIT_FALSE_START:
167     MOV game_state, #ST_FAIL
168     MOV beep_count, #BEEP_LONG
169     RET
170 WAIT_DONE:
171     MOV game_state, #ST_SHOW
172     RET
173
174 STATE_SHOW:
175     LCALL MAKE_RANDOM
176     MOV A, rand_seed
177     ANL A, #0FH          ; raw random value 0..15
178     CLR C
179     SUBB A, #0AH          ; only use digit targets 0..9
180     JNC SHOW_TARGET_DIGIT_OK ; 10..15 are mapped to 0..5
181     ADD A, #0AH          ; restore 0..9 when borrow happened
182 SHOW_TARGET_DIGIT_OK:
183     MOV target_key, A
184     MOV disp3, #SEG_BLANK_IDX
185     MOV disp2, #SEG_BLANK_IDX
186     MOV disp1, #SEG_BLANK_IDX
187     MOV disp0, target_key
188     CLR EA                ; keep visible edge and timer start together
189     MOV scan_digit, #00H
190     LCALL DISPLAY_SCAN    ; drive target digit pins before timing starts
191     CLR TR2               ; restart 1 ms Timer2 at the visible target edge
192     MOV TMR2L, #TIMERO_RELOAD_L
193     MOV TMR2H, #TIMERO_RELOAD_H
194     CLR TF2L
195     CLR TF2H
196     MOV react_lo, #00H
197     MOV react_hi, #00H
198     MOV timing_flag, #01H
199     SETB TR2
200     SETB EA
201     MOV beep_count, #BEEP_SHORT
202     MOV led_pattern, #07FH
203     LCALL LED_WRITE_A
204     MOV game_state, #ST_PLAY

```

```

205         RET
206
207 STATE_PLAY:
208     MOV A, game_mode
209     CJNE A, #MODE_RHYTHM, STATE_REACTION_PLAY
210     LCALL STATE_RHYTHM_PLAY
211     RET
212
213 STATE_REACTION_PLAY:
214     ; Fixed timeout: about 1800ms.
215     MOV A, react_hi
216     CJNE A, #07H, PLAY_READ_KEY
217     MOV A, react_lo
218     CLR C
219     SUBB A, #008H          ; 7*256+8=1800
220     JNC PLAY_TIMEOUT
221
222 PLAY_READ_KEY:
223     MOV A, key_value
224     CJNE A, #KEY_NONE, PLAY_HAS_KEY
225     RET
226
227 PLAY_HAS_KEY:
228     ; Capture reaction time immediately at the first detected press.
229     ; Then debounce-confirm the same key. The displayed time remains
230     ; the first-detection time, not the post-debounce time.
231     MOV last_key, A
232     CLR ET2
233     MOV score_lo, react_lo
234     MOV score_hi, react_hi
235     SETB ET2
236     LCALL DEBOUNCE_DELAY
237     LCALL SCAN_KEY_RAW
238     CJNE A, last_key, PLAY_DEBOUNCE_REJECT
239     MOV timing_flag, #00H
240     MOV A, last_key
241     CJNE A, target_key, PLAY_WRONG
242     LCALL UPDATE_BEST
243     LCALL SHOW_REACTION_TIME
244     MOV beep_count, #BEEP_SHORT
245     MOV game_state, #ST_SUCCESS
246     MOV show_timer, #160
247     LCALL UART_SEND_SCORE
248     RET
249
250 PLAY_DEBOUNCE_REJECT:
251     RET
252
253 PLAY_WRONG:
254
255 PLAY_TIMEOUT:
256     MOV timing_flag, #00H
257     MOV game_state, #ST_FAIL
258     MOV show_timer, #120
259     MOV beep_count, #BEEP_LONG
260     LCALL UART_SEND_FAIL
261     RET
262
263 STATE_RHYTHM_PLAY:
264     MOV A, key_value

```



```

261         CJNE A, #KEY_NONE, RHYTHM_HAS_KEY
262         RET
263 RHYTHM_HAS_KEY:
264         MOV last_key, A
265         LCALL DEBOUNCE_DELAY
266         LCALL SCAN_KEY_RAW
267         CJNE A, last_key, RHYTHM_DEBOUNCE_REJECT
268         MOV A, last_key
269         CJNE A, #00H, RHYTHM_WRONG
270         MOV A, led_pattern
271         CJNE A, rhythm_pat, RHYTHM_WRONG
272 RHYTHM_HIT:
273         INC rhythm_score
274         LCALL SHOW_RHYTHM_SCORE
275         MOV beep_count, #BEEP_SHORT
276         LCALL WAIT_MATRIX_RELEASE
277         MOV A, rhythm_score
278         CLR C
279         SUBB A, #20
280         JC RHYTHM_CONTINUE
281         MOV game_state, #ST_SUCCESS
282         MOV led_pattern, #00H
283         LCALL LED_WRITE_A
284         RET
285 RHYTHM_CONTINUE:
286         RET
287 RHYTHM_DEBOUNCE_REJECT:
288         RET
289 RHYTHM_WRONG:
290         MOV game_state, #ST_FAIL
291         MOV beep_count, #BEEP_LONG
292         LCALL UART_SEND_FAIL
293         RET
294
295 STATE_SUCCESS:
296         MOV led_pattern, #00H      ; all LEDs on, active-low
297         LCALL LED_WRITE_A
298         RET
299
300 STATE_FAIL:
301         MOV disp3, #0FH           ; F
302         MOV disp2, #0EH           ; E
303         MOV disp1, #SEG_R_IDX     ; r
304         MOV disp0, #SEG_R_IDX     ; r
305         MOV led_pattern, #OFFH
306         LCALL LED_WRITE_A
307         RET

```

Listing 8: isr.asm

```

1 ;-----
2 ; isr.asm - Timer0/Timer2/INT0 interrupt services
3 ;-----
4 TIMERO_ISR:
5     PUSH ACC
6     PUSH PSW

```

```

7      MOV TH0, #TIMER0_RELOAD_H
8      MOV TLO, #TIMER0_RELOAD_L
9
10     LCALL DISPLAY_SCAN
11
12     MOV A, beep_count
13     JZ TO_BEEP_OFF
14     SETB P3.1
15     DEC beep_count
16     SJMP TO_TIME
17 TO_BEEP_OFF:
18     CLR P3.1
19
20 TO_TIME:
21     INC ms_sub
22     MOV A, ms_sub
23     ANL A, #1FH
24     JNZ TO_PWM
25     MOV A, game_state
26     CJNE A, #ST_WAIT, TO_PWM
27     MOV A, led_pattern
28     RL A
29     JNZ TO_LED_OK
30     MOV A, #0FEH
31 TO_LED_OK:
32     MOV led_pattern, A
33
34 TO_PWM:
35     LCALL LED_PWM_SERVICE
36
37 TO_EXIT:
38     POP PSW
39     POP ACC
40     RETI
41
42 ;-----
43 ; Timer2 ISR: exact 1 ms game time base, independent from display refresh work.
44 ; Timer2 auto-reloads in hardware, so ISR execution length does not stretch
45 ; the reaction-time tick.
46 ;-----
47 TIMER2_ISR:
48     PUSH ACC
49     PUSH PSW
50     CLR TF2H
51     CLR TF2L
52
53     MOV A, wait_tick
54     ORL A, wait_hi
55     JZ T2_SHOW_TIMER
56     MOV A, wait_tick
57     JNZ T2_DEC_WAIT_LO
58     MOV A, wait_hi
59     JZ T2_SHOW_TIMER
60     DEC wait_hi
61     MOV wait_tick, #OFFH
62     SJMP T2_SHOW_TIMER

```

```

63 T2_DEC_WAIT_LO:
64     DEC wait_tick
65
66 T2_SHOW_TIMER:
67     MOV A, show_timer
68     JZ T2_REACT
69     DEC show_timer
70
71 T2_REACT:
72     MOV A, timing_flag
73     JZ T2_RHYTHM
74     INC react_lo
75     MOV A, react_lo
76     JNZ T2_RHYTHM
77     INC react_hi
78
79 T2_RHYTHM:
80     MOV A, game_state
81     CJNE A, #ST_PLAY, T2_EXIT
82     MOV A, game_mode
83     CJNE A, #MODE_RHYTHM, T2_EXIT
84     MOV A, rhythm_tick
85     JZ T2_RHYTHM_ADVANCE
86     DEC rhythm_tick
87     SJMP T2_EXIT
88 T2_RHYTHM_ADVANCE:
89     MOV A, rhythm_diff
90     CJNE A, #DIFF_LOW, T2_RHYTHM_NOT_LOW
91     MOV rhythm_tick, #220
92     SJMP T2_RHYTHM_SPEED_DONE
93 T2_RHYTHM_NOT_LOW:
94     CJNE A, #DIFF_HIGH, T2_RHYTHM_NORMAL_SPEED
95     MOV rhythm_tick, #90
96     SJMP T2_RHYTHM_SPEED_DONE
97 T2_RHYTHM_NORMAL_SPEED:
98     MOV rhythm_tick, #160
99 T2_RHYTHM_SPEED_DONE:
100     MOV A, led_pattern
101     RL A
102     MOV led_pattern, A
103
104 T2_EXIT:
105     POP PSW
106     POP ACC
107     RETI
108
109 ;-----
110 ; INTO_ISR: not used. KINT is polled and debounced in the main loop.
111 ;-----
112 INTO_ISR:
113     RETI

```

6. 显示、键盘、LED 和 UART 模块

Listing 9: display.asm

```

1 ;-----
2 ; display.asm - 4-digit seven-segment display routines
3 ;-----
4 CLEAR_DISPLAY:
5     MOV disp0, #SEG_BLANK_IDX
6     MOV disp1, #SEG_BLANK_IDX
7     MOV disp2, #SEG_BLANK_IDX
8     MOV disp3, #SEG_BLANK_IDX
9     RET
10
11 DISPLAY_SCAN:
12     MOV A, scan_digit
13     ANL A, #03H
14     MOV scan_digit, A
15     JZ DISP_D0
16     CJNE A, #01H, DISP_NOT_D1
17     MOV A, disp1
18     LCALL LOAD_SEG_CODE
19     MOV P1, A
20     ANL P0, #03FH
21     ORL P0, #040H          ; A=1,B=0 -> digit 1
22     SJMP DISP_NEXT
23 DISP_NOT_D1:
24     CJNE A, #02H, DISP_D3
25     MOV A, disp2
26     LCALL LOAD_SEG_CODE
27     MOV P1, A
28     ANL P0, #03FH
29     ORL P0, #080H          ; A=0,B=1 -> digit 2
30     SJMP DISP_NEXT
31 DISP_D3:
32     MOV A, disp3
33     LCALL LOAD_SEG_CODE
34     MOV P1, A
35     ANL P0, #03FH
36     ORL P0, #0C0H          ; A=1,B=1 -> digit 3
37     SJMP DISP_NEXT
38 DISP_D0:
39     MOV A, disp0
40     LCALL LOAD_SEG_CODE
41     MOV P1, A
42     ANL P0, #03FH          ; A=0,B=0 -> digit 0
43 DISP_NEXT:
44     INC scan_digit
45     RET
46
47 LOAD_SEG_CODE:
48     ANL A, #1FH
49     MOV DPTR, #SEG_TABLE
50     MOVC A, @A+DPTR
51     RET
52
53 SHOW_REACTION_TIME:
54     ; Convert full 16-bit reaction time in score_hi:score_lo to 4 decimal digits.
55     ; This avoids the old low-byte-only display error for times above 255ms.

```

```

56         LCALL BIN16_TO_BCD4
57         MOV disp0, bcd_ones
58         MOV disp1, bcd_tens
59         MOV disp2, bcd_hund
60         MOV disp3, bcd_thou
61         RET
62
63 LOAD_RHYTHM_PATTERNS:
64         MOV A, rhythm_target
65         LCALL LOAD_RHYTHM_LED_PATTERN
66         MOV rhythm_pat, A
67         RET
68
69 LOAD_RHYTHM_LED_PATTERN:
70         ANL A, #07H
71         MOV DPTR, #RHYTHM_LED_TABLE
72         MOVC A, @A+DPTR
73         RET
74
75 SHOW_RHYTHM_SCORE:
76         MOV disp3, #SEG_L_IDX
77         MOV A, rhythm_target
78         INC A
79         MOV disp2, A
80         MOV bcd_tens, #00H
81         MOV A, rhythm_score
82         MOV bcd_ones, A
83 SRS_TENS_LOOP:
84         MOV A, bcd_ones
85         CLR C
86         SUBB A, #10
87         JC SRS_DONE
88         MOV bcd_ones, A
89         INC bcd_tens
90         SJMP SRS_TENS_LOOP
91 SRS_DONE:
92         MOV disp1, bcd_tens
93         MOV disp0, bcd_ones
94         RET
95
96 BIN16_TO_BCD4:
97         MOV bcd_thou, #00H
98         MOV bcd_hund, #00H
99         MOV bcd_tens, #00H
100        MOV bcd_ones, #00H
101        MOV conv_lo, score_lo
102        MOV conv_hi, score_hi
103 BCD_1000_LOOP:
104        CLR C
105        MOV A, conv_lo
106        SUBB A, #0E8H
107        MOV R0, A
108        MOV A, conv_hi
109        SUBB A, #03H
110        JC BCD_100_LOOP
111        MOV conv_hi, A

```

```

112         MOV conv_lo, R0
113         INC bcd_thou
114         SJMP BCD_1000_LOOP
115 BCD_100_LOOP:
116         MOV A, conv_lo
117         MOV bcd_ones, A
118 BCD_H_LOOP:
119         MOV A, bcd_ones
120         CLR C
121         SUBB A, #100
122         JC BCD_T_LOOP
123         MOV bcd_ones, A
124         INC bcd_hund
125         SJMP BCD_H_LOOP
126 BCD_T_LOOP:
127         MOV A, bcd_ones
128         CLR C
129         SUBB A, #10
130         JC BCD_DONE
131         MOV bcd_ones, A
132         INC bcd_tens
133         SJMP BCD_T_LOOP
134 BCD_DONE:
135         RET

```

Listing 10: keypad.asm

```

1  ;-----
2  ; keypad.asm - matrix keypad and KINT scanning
3  ;-----
4  SCAN_KEY:
5         LCALL SCAN_KEY_RAW
6         CJNE A, #KEY_NONE, SK_DEBOUNCE
7         RET
8  SK_DEBOUNCE:
9         MOV last_key, A
10        LCALL DEBOUNCE_DELAY
11        LCALL SCAN_KEY_RAW
12        CJNE A, last_key, SK_NONE
13        RET
14  SK_NONE:
15        MOV A, #KEY_NONE
16        RET
17
18  SCAN_KEY_RAW:
19        MOV R4, #00H           ; row index
20        MOV R5, #0FEH         ; active low row pattern
21  SCAN_ROW_LOOP:
22        MOV A, R5
23        ORL A, #0FOH
24        MOV P2, A
25        NOP
26        NOP
27        MOV A, P2
28        ANL A, #0FOH
29        CJNE A, #0FOH, KEY_FOUND

```

```

30      MOV A, R5
31      RL A
32      ORL A, #0F0H
33      MOV R5, A
34      INC R4
35      CJNE R4, #04H, SCAN_ROW_LOOP
36      MOV A, #KEY_NONE
37      RET
38
39 KEY_FOUND:
40      MOV R6, #00H          ; col index
41      JNB P2.4, KEY_COL_DONE
42      INC R6
43      JNB P2.5, KEY_COL_DONE
44      INC R6
45      JNB P2.6, KEY_COL_DONE
46      INC R6
47      JNB P2.7, KEY_COL_DONE
48      MOV A, #KEY_NONE
49      RET
50 KEY_COL_DONE:
51      MOV A, R6
52      RL A
53      RL A          ; column * 4
54      ADD A, R4
55      RET
56
57 WAIT_MATRIX_RELEASE:
58      LCALL SCAN_KEY_RAW
59      CJNE A, #KEY_NONE, WAIT_MATRIX_RELEASE
60      LCALL DEBOUNCE_DELAY
61      RET
62
63 ;-----
64 ; KINT scanning with debounce
65 ; KINT is on P0.1 and is low active.
66 ; Returns A=01H once per stable press, or A=00H if no valid press.
67 ; It also waits for release so one physical press cannot start multiple rounds.
68 ;-----
69 SCAN_KINT:
70      JB P0.1, KINT_NONE      ; high means not pressed
71      LCALL DEBOUNCE_DELAY
72      JB P0.1, KINT_NONE
73 KINT_WAIT_RELEASE:
74      JNB P0.1, KINT_WAIT_RELEASE
75      LCALL DEBOUNCE_DELAY
76      MOV A, #01H
77      RET
78 KINT_NONE:
79      MOV A, #00H
80      RET
81
82 DEBOUNCE_DELAY:
83      MOV R2, #40
84 DB_D1:      MOV R3, #200
85 DB_D2:      DJNZ R3, DB_D2

```

```

86         DJNZ R2, DB_D1
87         RET

```

Listing 11: led_bar.asm

```

1  ;-----
2  ; led_bar.asm - 74HCT164 LED output and PWM service
3  ;-----
4  LED_PWM_SERVICE:
5      INC led_pwm
6      MOV A, led_pwm
7      ANL A, #03H          ; 1/4 duty cycle, about 25% brightness
8      JNZ LED_PWM_OFF
9      MOV A, led_pattern
10     SJMP LED_PWM_WRITE
11 LED_PWM_OFF:
12     MOV A, #0FFH          ; active-low LEDs off
13 LED_PWM_WRITE:
14     PUSH 07H
15     LCALL LED_WRITE_A
16     POP 07H
17     RET
18
19 LED_WRITE_A:
20     MOV R7, #08H
21 LED_BIT_LOOP:
22     RLC A
23     MOV P3.3, C
24     SETB P3.4
25     NOP
26     CLR P3.4
27     DJNZ R7, LED_BIT_LOOP
28     RET

```

Listing 12: uart.asm

```

1  ;-----
2  ; uart.asm - UART output helpers
3  ;-----
4  UART_PUTCHAR:
5      JNB TIO, UART_PUTCHAR
6      CLR TIO
7      MOV SBUF0, A
8  UART_WAIT_SENT:
9      JNB TIO, UART_WAIT_SENT
10     RET
11
12 UART_SEND_CRLF:
13     MOV A, #0DH
14     LCALL UART_PUTCHAR
15     MOV A, #0AH
16     LCALL UART_PUTCHAR
17     RET
18
19 UART_SEND_STRING:
20     CLR A

```



```

21         MOV A, @A+DPTR
22         JZ  UART_STR_DONE
23         LCALL UART_PUTCHAR
24         INC DPTR
25         SJMP UART_SEND_STRING
26 UART_STR_DONE:
27         RET
28
29 UART_BANNER:
30         MOV DPTR, #MSG_BANNER
31         LCALL UART_SEND_STRING
32         LCALL UART_SEND_CRLF
33         RET
34
35 UART_SEND_START:
36         MOV DPTR, #MSG_START
37         LCALL UART_SEND_STRING
38         LCALL UART_SEND_CRLF
39         RET
40
41 UART_SEND_FAIL:
42         MOV DPTR, #MSG_FAIL
43         LCALL UART_SEND_STRING
44         LCALL UART_SEND_CRLF
45         RET
46
47 UART_SEND_SCORE:
48         MOV DPTR, #MSG_SCORE
49         LCALL UART_SEND_STRING
50         MOV A, score_hi
51         LCALL UART_SEND_HEX
52         MOV A, score_lo
53         LCALL UART_SEND_HEX
54         MOV DPTR, #MSG_MS
55         LCALL UART_SEND_STRING
56         LCALL UART_SEND_CRLF
57         RET
58
59 UART_SEND_HEX:
60         MOV uart_temp, A
61         SWAP A
62         ANL A, #0FH
63         LCALL UART_HEX_NIBBLE
64         MOV A, uart_temp
65         ANL A, #0FH
66         LCALL UART_HEX_NIBBLE
67         RET
68
69 UART_HEX_NIBBLE:
70         ADD A, #030H
71         CJNE A, #03AH, HEX_CHECK
72 HEX_CHECK:
73         JC  HEX_OUT
74         ADD A, #07H
75 HEX_OUT:
76         LCALL UART_PUTCHAR

```

77

RET

7. 辅助逻辑与查找表

Listing 13: random_score.asm

```

1 ;-----
2 ; random_score.asm - random seed and best-score helpers
3 ;-----
4 MAKE_RANDOM:
5     MOV A, rand_seed
6     XRL A, TLO
7     RL A
8     ADD A, #03DH
9     MOV rand_seed, A
10    RET
11
12 UPDATE_BEST:
13     ; smaller reaction time is better. best initialized to FFFF.
14     MOV A, score_hi
15     CLR C
16     SUBB A, best_hi
17     JC BEST_UPDATE
18     JNZ BEST_DONE
19     MOV A, score_lo
20     CLR C
21     SUBB A, best_lo
22     JC BEST_UPDATE
23     SJMP BEST_DONE
24 BEST_UPDATE:
25     MOV best_hi, score_hi
26     MOV best_lo, score_lo
27 BEST_DONE:
28     RET

```

Listing 14: tables.asm

```

1 ;-----
2 ; tables.asm - ROM lookup tables and UART strings
3 ;-----
4 RHYTHM_LED_TABLE:
5     DB OFEH,OFDH,OFBH,OF7H,0EFH,ODFH,0BFH,07FH
6
7 SEG_TABLE:
8     DB 0FCH,060H,0DAH,0F2H,066H,0B6H,0BEH,0EOH
9     DB OFEH,0E6H,0EEH,03EH,09CH,07AH,09EH,08EH
10    DB 000H,002H,00AH,01CH
11
12 MSG_BANNER:
13    DB 'C8051F310 Reaction Game Ready',00H
14 MSG_START:
15    DB 'START',00H
16 MSG_SCORE:
17    DB 'SCORE HEX(ms)=' ,00H

```

```
18 MSG_MS:
19         DB ' ms',00H
20 MSG_FAIL:
21         DB 'FAIL',00H
```